



Mapless autonomous navigation for UGV in cluttered off-road environment with the guidance of wayshowers using deep reinforcement learning

Zhijian Li¹ · Xu Li¹ · Jinchao Hu² · Xixiang Liu¹

Accepted: 6 September 2024 / Published online: 2 January 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

Navigating unmanned ground vehicle (UGV) through off-road environments is critical for various tasks like exploration and rescue. Unlike scenarios allowing offline global planning based on prior knowledge, online navigation becomes essential due to the dynamic nature of these tasks. Although deep reinforcement learning (DRL) offers promise for mapless autonomous navigation due to its end-to-end advantages, existing approaches often rely solely on goal positions. This neglects the complex distribution of obstacles along the path, leading to inefficient interactions with the environment during training. To address this challenge, a deep reinforcement learning framework is proposed for autonomous navigation guided by wayshowers. Initially, a new metric is developed based on multilevel analysis to generate elevation maps, aiding in the identification of optimal wayshowers. Upon integrating wayshower information with other inputs, a multi-head attention (MHA) module is incorporated into DRL network, which includes a length attention mechanism to enhance focus on recent historical observation sequences to promote model convergence. Furthermore, the reward function is reshaped to offer dense reward signals, thereby resolving the sparse reward problem inherent in goal-driven methods. To validate the proposed approach, experiments are conducted on several off-road maps in both the Carla and Gazebo simulators. The results demonstrate the superiority of our method not only in simple environments but also in more challenging scenarios.

Keywords Mapless autonomous navigation · Unmanned ground vehicle · Deep reinforcement learning · Wayshower

1 Introduction

Autonomous navigation plays a pivotal role in various UGV scenarios, including exploration, mapping, rescue missions, and inspection [1]. Given the inherent uncertainty in these applications, developing robust autonomous navigation systems is crucial for mission success and human safety.

Traditional motion planners face significant challenges in navigating off-road environments due to the diverse and unpredictable nature of obstacles in terms of location, size, and shape. While approaches like state lattice have been introduced to tackle these challenges [2–4], they often rely on pre-existing maps obtained through techniques like Simultaneous Localization and Mapping (SLAM) [5, 6]. However, acquiring such prior knowledge in unknown off-road environments remains a daunting task. Hence, there is a pressing need to develop mapless navigation strategies that rely solely on sensor input and goal information.

Reinforcement learning algorithms, renowned for their capacity to represent high-dimensional states and superior data efficiency, have garnered increasing interest in the UGV community, particularly in autonomous driving [7]. End-to-end control algorithms based on deep reinforcement learning have demonstrated human driver-like behavior by directly mapping sensory information to vehicle control signals, enabling control of steering angle and throttle to accomplish driving tasks. Extensive research has been conducted

✉ Xu Li
101010791@seu.edu.cn

Zhijian Li
220223271@seu.edu.cn

Jinchao Hu
jinchaohu@nxu.edu.cn

Xixiang Liu
scliuseu@163.com

¹ School of Instrument Science and Engineering, Southeast University, Nanjing 210096, China

² School of Information Engineering, Ningxia University, Yinchuan 750021, China

on DRL-based autonomous driving, spanning indoor navigation [8–10], urban autonomous driving [11–14], and off-road navigation [15–17].

Traditional DRL approaches for navigation tasks typically adopt a goal-driven strategy, where the reward function incentivizes the UGV to move closer to the goal. This often involves designing the reward function to allocate higher rewards when the UGV approaches the goal with its head oriented towards it. However, these methods can oversimplify the navigation task and overlook the diverse nature of obstacles, leading to sparse reward problems and inefficient data exploration during training [18].

To address these challenges, a mapless navigation framework is proposed (see Fig. 1) that incorporates wayshowers to guide the UGV's motion. This approach utilizes continuous inputs including local elevation maps, UGV states, goal-related vectors, and information from wayshowers to output actions for steering and throttle control. Key contributions of our approach include: 1) Introducing a new method to generate elevation maps and select the optimal wayshower based on a cost grid map derived from lidar point clouds using multi-level analysis. This method considers UGV states, goal information, and the surrounding

environment, enabling the optimal wayshower to guide the UGV along a collision-free path towards the goal efficiently; 2) Utilizing optimal wayshowers during training to design a dense and balanced reward function that integrates goal and wayshower influences, thereby mitigating the sparse reward problem common in goal-driven methods and enhancing navigation in challenging environments; 3) Enhancing the DRL network with a MHA module and a length attention mechanism, allowing the model to focus more on recent historical observation sequences, thereby promoting faster convergence and improved decision-making.

2 Related work

Recent advancements in autonomous navigation have made significant strides, particularly in addressing challenges posed by cluttered environments and the limited perceptual range of egocentric observations. Despite these advancements, persistent shortcomings continue to drive ongoing research efforts in this field. This section aims to highlight key developments, examine their limitations, and elucidate how the proposed approach distinguishes itself.

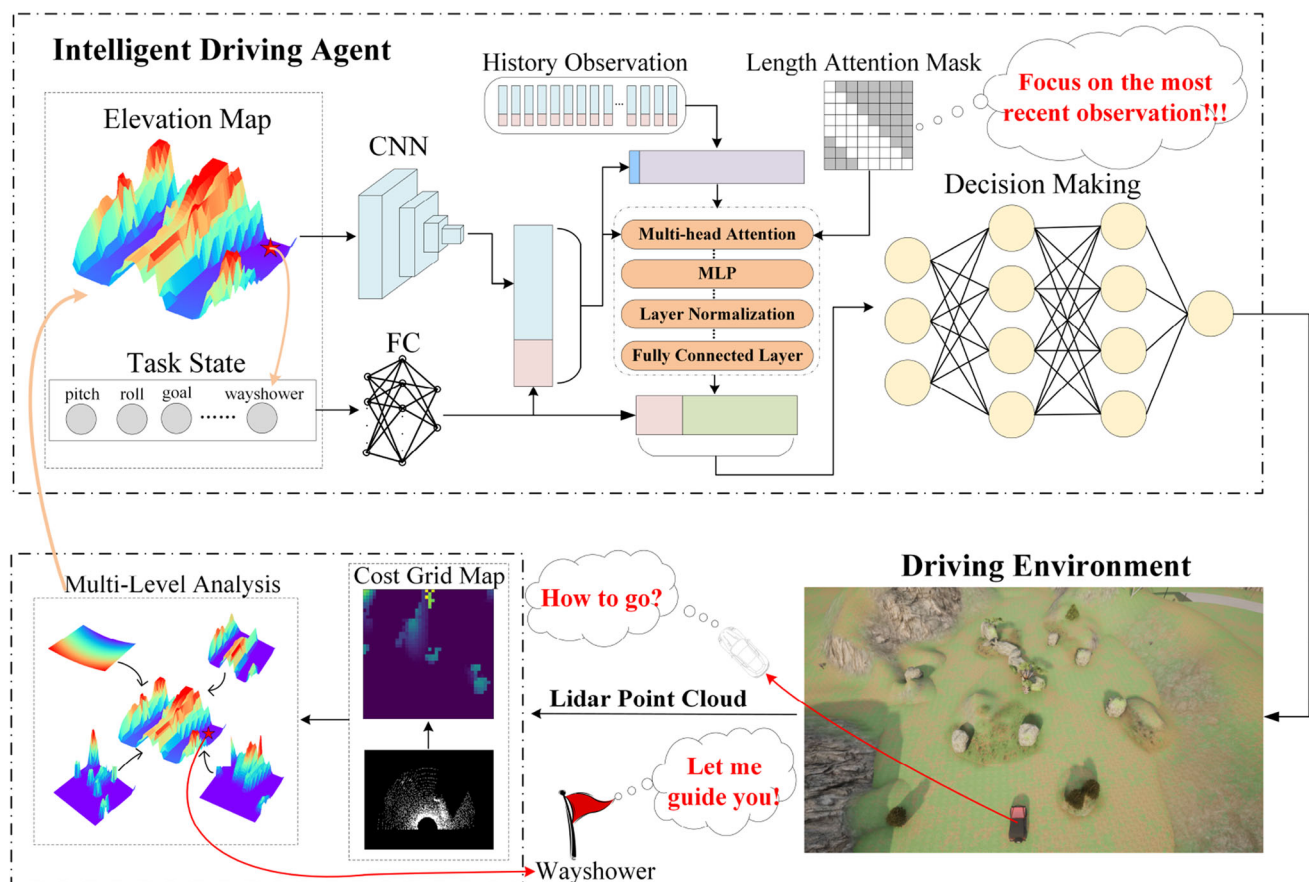


Fig. 1 Framework overview of the proposed system

2.1 Goal driven DRL-based autonomous navigation

The early use of DRL to tackle the UGV navigation challenge in cluttered environments with unknown rough terrain was explored in [19]. In this study, a DRL network was developed and trained using raw sensory data collected from onboard sensors. The network aimed to determine a series of local navigation actions for the UGV to execute. Similarly, [16] introduced a sim-to-real pipeline that taught a UGV to navigate through 3D cluttered real-world terrains featuring abrupt changes in surface normal and elevations. This approach aimed to bridge the gap between simulated and real-world environments, enabling effective navigation despite challenging terrain conditions. In another study, [17], RGB images and point clouds were leveraged to train a multi-modal fusion network. This network was designed to address the problem of autonomous navigation in complex environments such as collapsed cities or natural caves. However, as described in the introduction, these studies directly defined the reward function based on the straight line length from the UGV to the final goal, while ignoring the possible effects of obstacles between the UGV and the final goal. This goal-driven strategy could lead to sparse rewards and data inefficiency due to unnecessary interactions with the environment during training, and potentially result in navigation failure.

2.2 Sub-goal guided DRL-based autonomous navigation

In [18], a hierarchical DRL framework for navigation tasks was introduced. This framework initially utilized historic laser data to update a grid occupation map using the SLAM algorithm. Subsequently, it generated a global path from the UGV to the final goal using global path planners. By iteratively moving close to each waypoint along the global path, the UGV successfully reached the target. Differing from [18, 20] introduced a strategy where waypoints were generated during local planning and updated once the UGV reached each one until reaching the target goal. Similarly, in [21], least-cost waypoints were generated on a cost-map encoding elevation information and terrain flatness using an attention mask. A dynamically feasible trajectory was computed using another DRL-based method. These methods also moved the UGV towards the target by sequentially approaching waypoints, with waypoints remaining unchanged until reached by the UGV. Unlike the aforementioned methods, the approach proposed in this work does not treat wayshowers as waypoints that must be reached. Instead, wayshowers are utilized to provide directional guidance and are considered part of the observation state used to train the network to navigate around obstacles and move closer to the goal position. This

approach allows for more adaptive navigation behavior, as the network can dynamically adjust its trajectory based on real-time observations rather than rigidly adhering to predetermined waypoints.

2.3 Memory-based DRL autonomous navigation

Utilizing implicit memory, such as recurrent neural networks (RNNs), to aggregate observations is a common practice in navigation tasks [11, 22]. However, these approaches encounter optimization challenges when dealing with long sequences. To address this issue, the attention mechanism has been introduced to visual navigation [23, 24] and autonomous driving [25, 26], enabling the exploration of spatio-temporal dependencies within aggregated observations. The use of a causal mask ensures that during training, each token in the sequence can only attend to tokens that occur before it, ensuring the model has access to past information but not future information. However, in navigation tasks, the importance of historical tokens diminishes over time, and decisions are often more influenced by recent observations. Therefore, long-term historical tokens may have little impact on current decisions. In this work, a transformer encoder is employed and a length mask mechanism is introduced to ensure that attention is focused on a fixed length of historical observation sequence. This mechanism allows the model to prioritize recent observations, thus improving its ability to make decisions based on relevant information while navigating.

2.4 Reward shaping strategy in DRL autonomous navigation

Reward shaping strategies have proven effective in various DRL navigation tasks. For instance, in [27], a reward shaping technique is employed to enhance generalization, encouraging the agent to follow intermediate strategies that mitigate the risk of being trapped in a local minimum. Similarly, in [28], an intervention-based reward shaping term is defined to provide negative feedback when a human intervenes in the navigation task. This feedback indicates that the agent is likely approaching failure or a local optimum, prompting corrective action. In [29], objective-oriented reward functions are designed to address the sparse reward problem and accelerate convergence rate. These functions include approaching the target, avoiding obstacles, and step rewards. In this work, the sparse reward problem commonly encountered in goal-driven methods is tackled by introducing a novel term related to wayshowers and a hyperparameter. These components are designed to guide the UGV to avoid obstacles and move closer to the goal position. This approach aims to provide more informative feedback to the agent during training, facilitating more efficient learning and navigation.

3 Methodology

The proposed method involves three major stages: 1) computing a navigation cost-map and selecting the optimal wayshower, 2) introducing state encoder and length attention mechanism, 3) reshaping reward and designing DRL network.

3.1 Wayshower selection

To select an optimal wayshower from the cost grid map, a heuristic function is developed to score all available candidate points and choose the best one with the lowest risk. This heuristic function takes into account various factors including the state of the UGV, the cost grid map, and the goal information. Specifically, the distance between the current position and the goal position, the presence of obstacles between the current position and the candidate point, the presence of obstacles between the candidate point and the goal position, and the terrain characteristics around the candidate points are considered. In this basic form, the heuristic score $h_{(i,j)}$ for candidate point $c_{(i,j)}$ in the cost map can be calculated as follows:

$$h_{(i,j)} = d_{(c_{(i,j)},g)} + h_{(c_{(i,j)},p_t)} + h_{(c_{(i,j)},g)} + I_{c_{(i,j)}} \quad (1)$$

where (i, j) is the candidate coordinate on the cost grid map, $d_{(c_{(i,j)},g)}$ is the Euclidean distance between the candidate points and the goal g , $h_{(c_{(i,j)},p_t)}$ is the risk for driving the UGV to the candidate point, p_t is the position of UGV at time t , $h_{(c_{(i,j)},g)}$ is the risk for driving the UGV from the candidate point to the goal and $I_{c_{(i,j)}}$ is the terrain elevation around the candidate points getting from the cost map.

The cost grid map is generated from LiDAR data, normalized based on the maximum and minimum heights observed. Before generating the grid map, patchwork [30] is employed to remove the outliers and ground points. The resulting cost grid map has dimensions of 40×40 , with each cell representing the maximum normalized height of LiDAR points within that grid. The map covers a range of 20 meters to the left and right of the UGV and 20 meters in front of it, ensuring adequate representation of the terrain and obstacles for effective navigation planning. Each grid cell measures 1 meter in the x-axis direction and 0.5 meters in the y-axis direction.

Given the primary objective of guiding the UGV to the goal, it is crucial to select a wayshower that is as close to the goal as possible. The Euclidean distance from each candidate wayshower point to the goal is calculated to prioritize points closer to the goal, facilitating more efficient navigation towards the destination.

$$d_{(c_{(i,j)},g)} = \text{sqrt}((0.5(i - i_g))^2 + (j - j_g)^2) \quad (2)$$

where (i_g, j_g) is the goal coordinate on the cost grid map.

In areas distant from the UGV, the density of LiDAR points may be low, resulting in grids with no LiDAR points or grids behind obstacles. These situations are marked as the lowest, and their influence on the selection of the wayshower is considered by introducing two functions:

$$h_{(c_{(i,j)},p_t)} = \sum E_N^t(B(p_t)) \quad (3)$$

$$h_{(c_{(i,j)},g)} = \sum E_N^t(\sum_{u,f \leq 40} (B(g))) \quad (4)$$

In the given equation, $B(p_t) = \sum \text{Bresenham}(c_{(u,f)}, p_t)$ calculates the grid coordinates between candidate points and the current position using the Bresenham algorithm. $h_{(c_{(i,j)},p_t)}$ measures the traversability of the path from the current position to the candidate, and E_N^t represents the normalized cost grid map. Similarly, $h_{(c_{(i,j)},g)}$ serves a similar purpose to $h_{(c_{(i,j)},p_t)}$, but only considers points inside the cost map calculated by the Bresenham algorithm. This restriction ensures that their coordinates u and f do not exceed 40, as the goal may be far from the current position and outside the sensing range. Furthermore, it is crucial for the terrain around the optimal wayshower to be as flat as possible:

$$I_{c_{(i,j)}} = e^{\frac{\sum_{b=-k}^k \sum_{w=-k}^k E_N^t(i+b, j+w)}{4k^2}} \quad (5)$$

where k is set 4, e is the Euler's number and $I_{c_{(i,j)}}$ is calculated using the exponential function to inflate the score difference between flat terrain and those with obstacles.

After combining these scoring methods, the final heuristic scoring function $h_{(i,j)}$ for evaluating candidate points is obtained. This function initially eliminates points if there are obstacles on the way from the current position to them, as well as from them to the goal position. Simultaneously, candidates closer to the goal position are preferred. Additionally, the selection is influenced by the flatness of the area around the candidate point. The task of selecting the optimal candidate point ultimately boils down to selecting the one with the minimal heuristic score:

$$\text{Index} = \text{UnravelIndex}(\text{argmin}(h)) \quad (6)$$

where $\text{UnravelIndex}(\text{argmin}(h))$ is used to find the index of the smallest value in the array $h = \sum_{i,j \leq 40} h_{(i,j)}$.

3.2 State encoder

The encoding of observation states involves two stages, as illustrated in Fig. 2, in the first stage, a pair of 2D convolutional layers with dimensions $40 \times 40 \times 8$ and $38 \times 38 \times 4$ respectively are utilized to process the input normalized elevation map h (with $n = 40$). Subsequently, the output feature

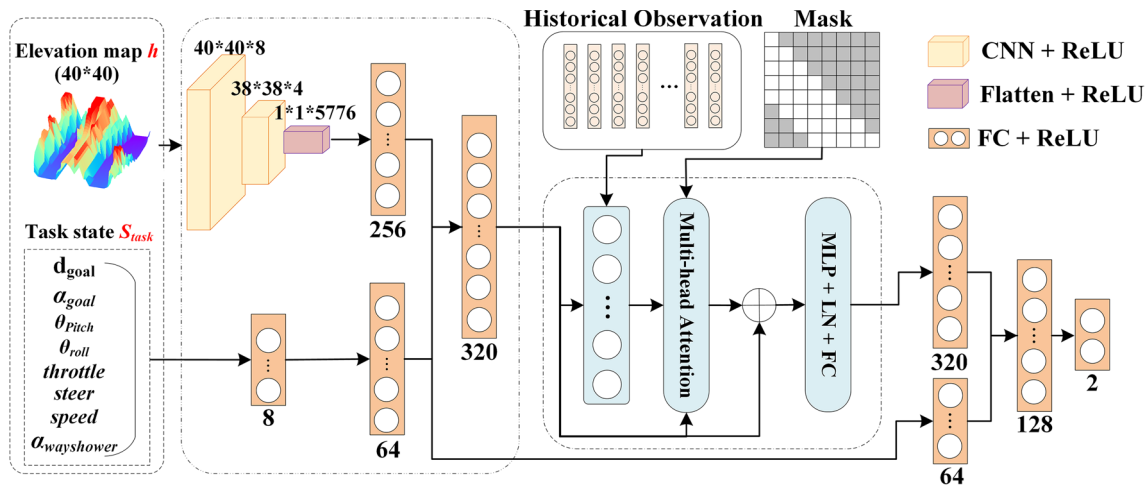


Fig. 2 The network of the proposed method

vector is flattened into a 1D vector with a size of $1 \times 1 \times 5776$. This vector is then concatenated with other task inputs S_{task} using fully connected (FC) layers. This process yields the low-dimensional state S_t . ReLU activation is employed in the hidden layers, as well as at the output layer.

$$S_t = \text{CONCAT}(\varphi_{CNN}(h), \varphi_{FC}(S_{task})) \quad (7)$$

The schematic diagram of the task states is illustrated in Fig. 3. Here, d_{goal} denotes the distance to the goal, α_{goal} represents the direction of the goal relative to the UGV, and $\alpha_{wayshower}$ signifies the direction of the wayshower relative to the UGV. Additionally, θ_{pitch} and θ_{roll} denote the pitch and roll angles, respectively, while *throttle*, *steer*, and *speed* correspond to the throttle, steering, and speed of the UGV.

In the second stage, a MHA module is employed to extract additional information from the historical observation sequence. The masked attention mechanism, as detailed in [31], can be formulated as a function Att that takes the query $Q \in \mathbb{R}^{(S_1 \times d_k)}$, key $K \in \mathbb{R}^{(S_2 \times d_k)}$, value $V \in \mathbb{R}^{(S_2 \times d_v)}$,

and mask $M \in \mathbb{R}^{(S_1 \times S_2)}$ as input. Here, d_k and d_v denote the dimensions of the query and value respectively, and M is the matrix added to the dot product of the queries and keys. The output of the Att function consists of S_1 elements of dimension d_v .

$$Att(Q, K, V, m) = (\text{softmax}(m + QK^T))V \quad (8)$$

As depicted in Fig. 4, instead of directly utilizing the low-dimensional encoded latent state, the input of the MHA module at time t is modified to include both the current state $S_t \in \mathbb{R}^{(1 \times d_1)}$ and the historical observation sequence $H_t = (S_t, S_{t-1}, S_{t-2}, \dots, S_{t-L+1}, S_{t-L}) \in \mathbb{R}^{(L \times d_1)}$. This approach allows for the exploration of information from the historical observation sequence. The rationale behind this modification is that the motion of the UGV depends not only on the current state but also on past observations. Leveraging historical information enables the network to better comprehend the evolution of the surrounding environment, thus facilitating more accurate decision-making. Addition-

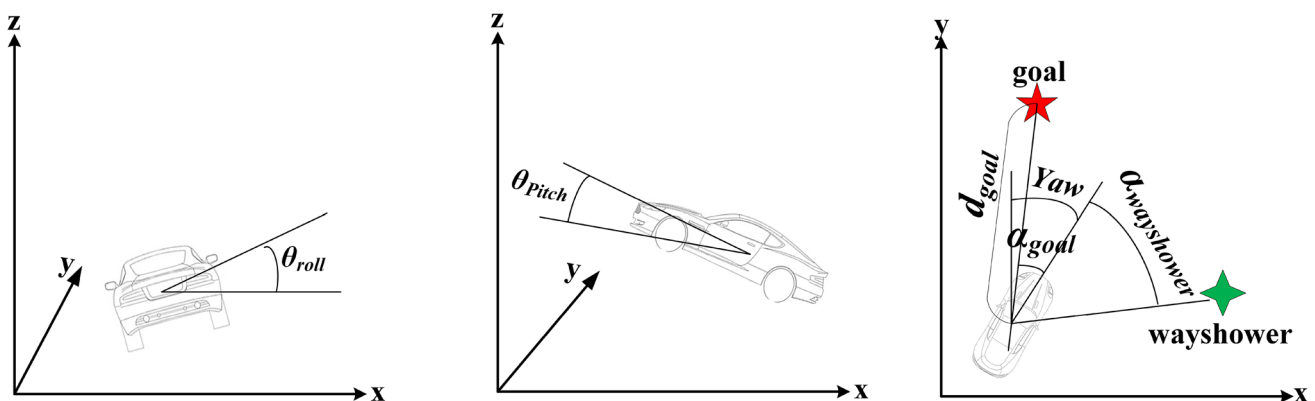


Fig. 3 The schematic diagram of the task states except speed, steer and throttle

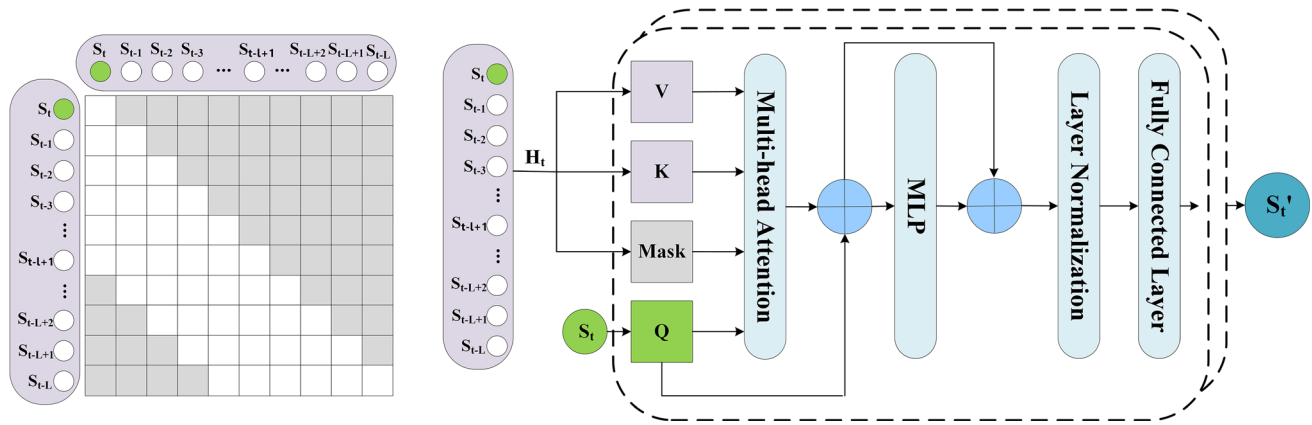


Fig. 4 Length mask and attention module block

ally, besides utilizing the causal mask, the length mask is introduced to ensure that attention is focused on a fixed length of the historical observation sequence. In this work, this means that driving experience from the most recent $l = 32$ timesteps is prioritized.

$$\text{Mask}(S_t, H_t) = \sum_{i,j=0}^{L-1} \text{mask}_{i,j} \quad (9)$$

where $\text{mask}_{i,j} = -\text{inf}$ if $j \geq i$ or $i - j \geq l + j$ (shaded ones in Fig. 4), else $\text{mask}_{i,j} = 0$ (whited ones in Fig. 4). $L = 512$ is the length of historical observation memory.

The casual mask is represented by an upper triangular matrix where the upper part is set to negative infinity, effectively masking out future tokens. This prevents the model from accessing future information during training, ensuring causality by preventing the model from cheating by attending to tokens that come after the current token. Similarly, the length mask is a lower triangular matrix shifted down by a certain number of rows ($l = 32$ in this work), where the lower part (excluding the diagonal) is also set to negative infinity. The length mask restricts the attention of each token to a local neighborhood around itself in the sequence. By limiting attention to a local region, the model is encouraged to learn dependencies within a fixed range. This focus on local dependencies can be advantageous for navigation tasks where long-range dependencies are less important, as it facilitates inference and boosts learning speed. The MHA module also includes the multi-layer perceptron (MLP), the fully connected layer, the layer normalization operation (LN), and the residual connections. Therefore, it can be described as:

$$\text{AttModule}(S_t, H_t) = \text{FC}(\text{LN}(\text{MLP}(S_t, \text{Att}(Q', K', V', \text{Mask}(S_t, H_t))), S_t)) \quad (10)$$

$$S'_t = \text{AttModule}(\text{AttModule}(S_t, H_t), H_t) \quad (11)$$

where $Q' = S_t W^Q \in R^{(1 \times d_k)}$, $K' = H_t W^K \in R^{(l \times d_k)}$, $V' = H_t W^V \in R^{(l \times d_v)}$. To address the constraints imposed by limited computational resources and the need for a lightweight architecture, the design employs a configuration comprising two encoder blocks, each block contains only four heads and the output of each AttModule serves as the query for the subsequent one, facilitating information flow.

3.3 RL network

The RL problem is defined as a Partially Observable Markov Decision Process (POMDP). This is formalized as a tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S} = \{S'\}$ denotes a set of states S' obtained from state encoder. $\mathcal{A} = \{a\}$ denotes a set of actions. $\mathcal{P} : S'_t \times a \rightarrow S'_{t+1}$ is the transition function. $\mathcal{R} : S' \times a \rightarrow r$ is the reward function. $\gamma \in [0, 1]$ is the discount factor. At each discrete timestep t , the RL agent observes the current state $S'_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$, receives a reward $r(S'_t, a_t)$ upon executing the action, and transitions to a new state $S'_{t+1} \in \mathcal{S}$.

The objective of autonomous navigation is to demonstrate collision-free path planning to reach the goal position with the guidance of wayshowers. Hence, the reward function is meticulously crafted to balance the influence of both the goal point and wayshowers on navigation. This function is designed to provide dense rewards to enhance the convergence efficiency of network. Specifically, the overall payoff comprises six individual components, outlined as follows:

$$r = \frac{r_d + r_h + r_p + r_a + r_t + r_w}{6} \quad (12)$$

where r_d terms heuristic reward, r_h is the heading reward, r_p determines pose reward, r_a denotes action reward, r_t represents terminal reward and r_w indicates reward for following

the wayshowers. The heuristic reward r_d is designed to motivate the UGV to move toward the goal position:

$$r_d = \eta \times m_d \times \|p_{t-1}^{(x,y)} - p_g^{(x,y)}\|_2 - \|p_t^{(x,y)} - p_g^{(x,y)}\|_2 \quad (13)$$

where $m_d=10$ is a fixed weight and $p_t^{(x,y)}$ denotes the real-time position of the UGV, $p_g^{(x,y)}$ indicates an arbitrary location of the goal point. Most importantly, parameter η is used to balance the impact of goal point and wayshowers when driving the UGV to the goal position, it is set to be 0.5 when there are obstacles between it and the UGV according to elevation map and 1 at other time. Simultaneously, the heading error between UGV's orientation and the directional vector pointing to the goal position also plays an important role in guiding UGV to the goal position:

$$r_h = (-\eta) \times m_h \times (\arctan((p_g^y - p_t^y), (p_g^x - p_t^x)) - Yaw) \quad (14)$$

where $m_h = 1/180$ and Yaw terms the heading angle of the UGV. Similarly, the action reward is designed to drive the UGV on comparatively even road, the UGV should remain stable posture, which means the pitch and roll should be close to 0° :

$$r_p = m_p \times (\cos(\theta_{pitch}) + \cos(\theta_{roll})) \quad (15)$$

where m_p is set to 0.5. Then the reward for following the guidance of wayshowers is designed as:

$$r_w = \eta \times (1 - \arctan(p_w^y, p_w^x)) \quad (16)$$

where (p_w^y, p_w^x) is the coordinate of wayshower relative to the UGV and can be calculated according to the index got from (6). Action reward is designed to drive the UGV at an expected speed:

$$r_a = \begin{cases} \frac{v}{v_{min}}, & \text{if } v < v_{min} \\ 1 - \text{abs}(\frac{v-v_{target}}{v_{max}-v_{target}}), & \text{if } v > v_{max} \\ 1, & \text{otherwise} \end{cases} \quad (17)$$

where abs is an absolute value operation and $v_{min} = 14 \text{ km/h}$, $v_{target} = 16 \text{ km/h}$, $v_{max} = 18 \text{ km/h}$ represent minimum speed, target speed and maximum speed respectively. The last but not least, reward for reaching goal position and collision penalty is designed as follows:

$$r_t = \begin{cases} -100, & \text{if collision} \\ 100, & \text{if } \tau \leq \tau_{th} \\ 0, & \text{otherwise} \end{cases} \quad (18)$$

where $\tau_{th} = 1.2 \text{ m}$ is the threshold that determines whether the UGV reaches the goal position.

By introducing novel terms, r_w and the hyperparameter η , into reward function, navigation efficacy is enhanced. The term r_w , relative to wayshowers, is designed to guide the UGV to avoid obstacles, while η is crafted to balance the influence of wayshowers and the final goal during training. This approach aims to provide more informative feedback to the UGV during training, facilitating more efficient learning and navigation.

Given the features extracted from the state encoder and reward function, the PPO algorithm [32] is chosen to learn the decision policy. In PPO algorithm, each step of the policy update is treated as an optimization problem. The algorithm incorporates the concept of importance sampling from the off-policy approach to formulate a new objective function known as the surrogate objective function. This objective function serves as a proxy for the actual objective, allowing for more stable and efficient updates of the policy.

$$\max_{\zeta} \mathbb{E}_{(S', a) \sim \pi_{\zeta_k}} \left[\frac{\pi_{\zeta_{k+1}}(S', a)}{\pi_{\zeta_k}(S', a)} A^{\pi_{\zeta_k}}(S', a) \right] \quad (19)$$

where π_{ζ} is the strategy in the scenario of autonomous driving, and $A^{\pi_{\zeta}}(S', a)$ is the estimate of the moment for the advantage function, which represents the advantage that can be obtained by taking the action a with the observation S' obtained from state encoder.

At each policy update, the algorithm optimizes ζ_k to obtain ζ_{k+1} using samples collected from the previous policy ζ_k and the corresponding estimated advantage function $A^{\pi_{\zeta_k}}(S', a)$. When updating the policy, it's crucial to ensure that the new policy ζ_{k+1} doesn't deviate too much from the previous policy ζ_k . This is achieved by constraining the divergence, which measures the dissimilarity between different probability distributions, between the new and old policies. Directly updating the policy objective function would lead to $\pi_{\zeta_{k+1}}$ deviating significantly from π_{ζ_k} . While this deviation could be mitigated by imposing a divergence constraint, doing so would make the policy update process more challenging. To address this issue and prevent excessive divergence in $\pi_{\zeta_{k+1}}$, the PPO algorithm introduces the Clipped Surrogate Objective:

$$\mathcal{L}^{CLIP}(\zeta) = \mathbb{E}_{(S', a) \sim \pi_{\zeta_k}} \left[\min \left(\frac{\pi_{\zeta}(a|S')}{\pi_{\zeta_k}(a|S')} A^{\pi_{\zeta_k}}(S', a), \text{clip} \left(\frac{\pi_{\zeta}(a|S')}{\pi_{\zeta_k}(a|S')}, 1-\epsilon, 1+\epsilon \right) A^{\pi_{\zeta_k}}(S', a) \right) \right] \quad (20)$$

The objective of this loss function is to maximize the approximate policy gradient, thereby making the probabilities of selecting actions under the new policy closer to those

that yield high rewards in the environment. In this work, the loss function is defined as:

$$\mathcal{L}(\zeta) = \mathcal{L}^{CLIP}(\zeta) + \text{entropy_coef} \times \mathbb{E}_{(S',a) \sim \pi_{\zeta_k}} [-\log(\pi_{\zeta_k}(S'|a))] \quad (21)$$

$\mathcal{L}(\zeta)$ incorporates the entropy of the policy as a regularization term to encourage exploration and prevent premature convergence to suboptimal policies. The term $\text{entropy_coef} \times \mathbb{E}_{(S',a) \sim \pi_{\zeta_k}} [-\log(\pi_{\zeta_k}(S'|a))]$ calculates the entropy of the policy, where entropy_coef is a hyperparameter typically set to 0.01. Entropy measures the uncertainty or randomness in the policy's action distribution. A higher entropy implies a more uniform distribution of probabilities across actions, indicating greater exploration. By including the entropy term in the training objective, the algorithm incentivizes policies that are not only effective but also diverse in their action selection. The value function is defined as:

$$\mathcal{L}(\phi) = \frac{1}{2} \mathbb{E}_{(S',a) \sim \pi_{\zeta_k}} [(V_{\phi_k}(S') + \text{clip}(V_{\phi_{k+1}}(S') - V_{\phi_k}(S'), -\delta, \delta)) - V_{\text{target}}(S')]^2] \quad (22)$$

Where $V_{\phi}(S')$ represents the predicted value by the Critic network for state S' , $V_{\text{target}}(S')$ denotes the actual obtained reward for state S' . The objective of this loss function is to minimize the discrepancy between the predicted values and the actual rewards obtained from the environment. By minimizing this difference, the Critic network learns to provide more accurate evaluations of the value of states, which is crucial for informing the policy's decision-making process in reinforcement learning tasks. The training algorithm is shown in Algorithm 1.

While it's true that PPO doesn't handle overestimation issues as comprehensively as SAC, which employs double Q-networks, it's essential to note that PPO can still achieve training stability through various techniques. For example, learning rate annealing gradually reduces the learning rate during training, allowing the model to converge more smoothly and potentially mitigating overestimation. Similarly, reward scaling adjusts the scale of rewards, which can help in balancing the learning process and reducing the likelihood of overestimation. Although these methods may not directly address the overestimation problem in the same way as double Q-networks, they have been shown to effectively stabilize training and improve overall performance in PPO-based systems [33–35].

4 Experiments

The proposed method undergoes training and evaluation within the CARLA simulator [36], renowned for its high-definition and open-source simulation platform tailored for autonomous driving research. This platform meticulously replicates driving environments and vehicle dynamics, incorporating raw sensor data inputs like camera RGB images and lidar point clouds via rendering and ray-casting techniques. Additionally, CARLA's exceptional 3D modeling capabilities facilitate the shaping of various terrains, enabling seamless experimentation and validation of the proposed approach. For experimentation purposes, four off-road maps are constructed, as depicted in Fig. 5. These maps vary in size: 30m × 60m, 50m × 100m, 50m × 100m, and 120m × 120m, respectively. Each map presents unique challenges, with Map (a) being the simplest, featuring minimal obstacles and slopes. In contrast, Maps (b) and (c) introduce

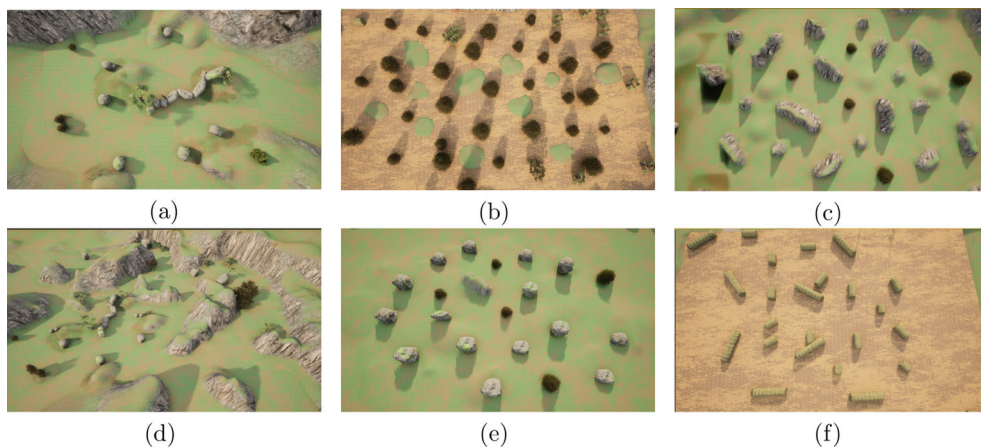


Fig. 5 Four off-road environment training maps and two validation maps in Carla simulation

Algorithm 1 Training Algorithm.

```

1: Initialize Carla Simulator
2: Initialize state encoder network parameters:  $\zeta$ 
3: Initialize actor and critic network parameters:  $\zeta, \phi$ .
4: Initialize historical observation:  $H \leftarrow \emptyset$ .
5: Initialize replay buffer  $\psi \leftarrow \emptyset$ .
6: Assign target parameters:  $\zeta_{target} \leftarrow \zeta, \phi_{target} \leftarrow \phi$ .
7: for episode: 1 to E do
8:   Initialize the environment state:  $s_t, s_{(task,t)} \sim \text{Env}$ 
9:   while not done do
10:    Calculate wayshower  $index_w$  from (6).
11:    Get encoding state  $s' = \text{AttModule}((s_t, s_{(task,t)}), H_t)$ .
12:    Sample an action:  $a_t \leftarrow \pi_{\zeta}(a_t | s')$ .
13:    Interact with the environment:  $r_t, s_{t+1}, index_w \sim \text{Env}$ .
14:    Store the state:  $\psi \leftarrow \psi \cup (s_t, s_{(task,t)}, a_t, r_t, s_{t+1}, s_{(task,t+1)})$ .
15:    if time to update actor-critic then
16:      for epoch: 1 to K do
17:        Compute loss and update actor-critic network.
18:        Update state encoder.
19:        Update target parameters:  $\zeta_{target} \leftarrow \zeta, \phi_{target} \leftarrow \phi$ .
20:      end for
21:      Clear replay buffer  $\psi \leftarrow \emptyset$ .
22:    end if
23:    Update the historical observation.
24:  end while
25: end for

```

a variety of obstacle sizes and types, denser obstacle layouts, and enlarged map sizes. Map (d) represents the most complex environment, featuring extensive obstacles, slopes, trees, and rocks, thus posing the greatest challenge for the UGV to navigate towards the goal position. The progressive complexity across these maps enables a comprehensive exploration of different algorithms' path planning capabilities across diverse scenarios. Maps (e) and (f) are used for model validation and differ from the first four training maps in terms of obstacle distribution, while their size is the same as that of map (b).

4.1 Experimental setups and implementation details

The Carla simulator is configured with specific settings for the experiment as follows. The agent vehicle selected is the '*vehicle.jeep*'. Environment perception is facilitated by the '*sensor.lidar.ray.cast*', which processes the lidar points cloud to generate the cost grid map for evaluation purposes. The lidar is positioned directly above the vehicle at a height of 2.4 meters, with attributes such as '*points_per_second*', '*horizontal_fov*', '*channels*', '*range*', '*rotation_frequency*', '*upper_fov*', and '*lower_fov*' set to 50000, 180, 64, 30, 10, 10, and -20, respectively. Additionally, the '*sensor.other.collision*' is employed to detect collisions during interactions within the training environment.

In the training stage, models undergo training across four distinct maps. Random goal positions and starting locations are selected from predefined pairs. Each episode terminates if the UGV encounters a collision, if the roll or pitch exceeds 25° , or if the timestep exceeds 1000. A successful navigation is achieved only when the UGV autonomously reaches the goal. Training procedures are conducted on a computer equipped with an Intel Core i7-12700 CPU, 64 GB of RAM, and an NVIDIA GTX 3080 Ti graphics card. The code is implemented using Python 3.7 and Pytorch 1.13.

4.2 Baseline methods and evaluation metrics

In the experiments, the proposed method is compared with six baseline methods as follows:

Far Planner [37]: This is a visibility graph-based traditional planning framework suitable for navigation tasks in known and unknown environments.

FASTER [38]: This is a traditional planner that obtains trajectories by having a safe back-up trajectory in the free-known space to enable optimization in both known and unknown spaces.

IDLE [20]: This is a state-of-the-art DRL method employs a strategy where waypoints were generated during local planning and updated once the UGV reached each one until reaching the target goal.

Terp [21]: This is another sub-goal guided state-of-the-art DRL method, least-cost waypoints were generated on a cost-map encoding elevation information and terrain flatness using an attention mask.

Benatti et al. [39]: This is a goal-driven DRL-based method which relies only on the RGB data and ensures that UGV can avoid colliding with obstacles.

Wang et al. [40]: This another goal-driven DRL-based method using top-down RGB data as input.

The utilization of the first two methods serves to strengthen the credibility of the proposed algorithm by showcasing its effectiveness when contrasted with traditional approaches. Through comparison with two goal-driven and two sub-goal guided methods, it aims to not only demonstrate the superior performance of the proposed approach but also highlight the limitations identified in the related work section pertaining to goal-driven and sub-goal guided methods. Furthermore, by examining two ablated versions of the proposed method, the specific impact of the techniques outlined in this article on performance enhancement can be assessed. Notably, the proposed model integrates length-masked attention and wayshowers, leading to its designation as LMAW.

For performance comparison, speed of convergence, length of planned paths, and driving stability are initially compared, followed by evaluation using **success rate (SR)**,

success weighted by path length (SPL), speed variation rate (SPVR), trajectory smoothness (TS), and goal heading deviation (GHD). SR represents the ratio of successful episodes to all episodes.

$$SR = \frac{1}{N} \sum_{i=1}^N Success_i \quad (23)$$

where $Success_i$ indicates the success in episode i using a binary code. SPL evaluates successes based on their adherence to the shortest path, calculated as:

$$SPL = \frac{1}{N} \sum_{i=1}^N Success_i \frac{len_i}{\max(p_i, len_i)} \quad (24)$$

where N denotes the number of testing episodes, len_i is the shortest path distance from the start point to the goal in episode i , p_i represents the length of the planned path. SPVR measures the absolute cumulative speed difference between the current timestep and the last timestep, given by:

$$SPVR = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} abs(v_{t+1} - v_t) \quad (25)$$

where T is the total timesteps for episode i and v_t is the speed of UGV at timestep t . TS typically refers to how smoothly a trajectory changes in terms of its curvature. It's a measure of how gradual or abrupt the changes in curvature are along a path, and it's important for understanding how comfortable or efficient a trajectory is.

$$TS = \frac{1}{N \times (T - 1)} \sum_{i=1}^N \sum_{t=1}^{T-1} \Delta \kappa_t \quad (26)$$

where κ_t is the curvature change of the t^{th} segment. GHD is a measure used to assess how well the UGV is aligning its heading or direction towards a desired goal or target direction over time. It quantifies the difference between the actual heading of the agent and the desired heading.

$$GHD = \frac{1}{N \times T} \sum_{i=1}^N \sum_{t=0}^{T-1} (\arctan((p_g^y - p_t^y), (p_g^x - p_t^x)) - Yaw) \quad (27)$$

4.3 Selection of wayshower

To validate the efficacy of the heuristic function utilized for selecting the optimal wayshower, various positions across the training map are examined, and the scoring functions are visualized. Specifically, all the components of (1) are

depicted in Fig. 6, with figures (b), (c), (d), and (e) corresponding to (2), (3), (4), and (5), respectively. Figure (a) presents the final elevation map with a height constraint below 200, where the point with the lowest score is preferred. Figure (b) highlights the prioritization of points closer to the goal in the distance evaluation. Additionally, figure (c) illustrates that points incur higher scores if obstacles lie between them and the goal, aligning with the criteria for candidate points and the UGV's current position depicted in figure (d). Furthermore, figure (e) demonstrates the rejection of points with uneven terrain surroundings. Ultimately, figure (a) integrates the advantages of (b), (c), (d), and (e), facilitating the selection of the optimal point using (6).

Four typical positions selected from the training maps are depicted in Fig. 7. The goal position is denoted by a red star, while the optimal wayshower is indicated by white stars. Each situation is represented with three images: the left image illustrates the UGV's location on the map, the middle image displays the cost map processed from the lidar point cloud, and the right image shows the corresponding elevation map. The results indicate that the heuristic evaluation function effectively selects the wayshower with the greatest potential to guide the UGV to the goal position. Furthermore, for comparison with sub-goal guided methods, performance will be further assessed in Section 4.4.

4.4 Hyperparameters optimization and model validation

Hyperparameter optimization and model validation are critical steps in the reinforcement learning that ensure models are both high-performing and generalizable, making them more reliable and effective when applied to untrained scenarios. The method outlined in [41] is adapted for this application, resulting in a similar two-stage process.

4.4.1 Hyperparameter optimization

In the initial stage, the hyperparameters—specifically the learning rate (lr), number of update epochs (n_epochs), and update step size (n_steps)—are optimized. The learning rate is either kept stable or linearly declined (from 0.0003 to 0.0001). The number of update epochs is then selected from the set [5, 10, 15, 20], and the update step size is chosen from [1024, 2048, 3072, 4096]. To prevent premature convergence during training, a two-step termination criterion is employed. If the primary goal of achieving a satisfactory reward is not met, a secondary criterion, the number of timesteps, which is capped at 1,000,000 steps, is used instead. For model testing, the trained model is evaluated across twenty episodes, with each episode utilizing one of the other three training maps shown in Fig. 5. For instance, if the model is trained on map (b), each evaluation episode will use one of the maps (a), (c),

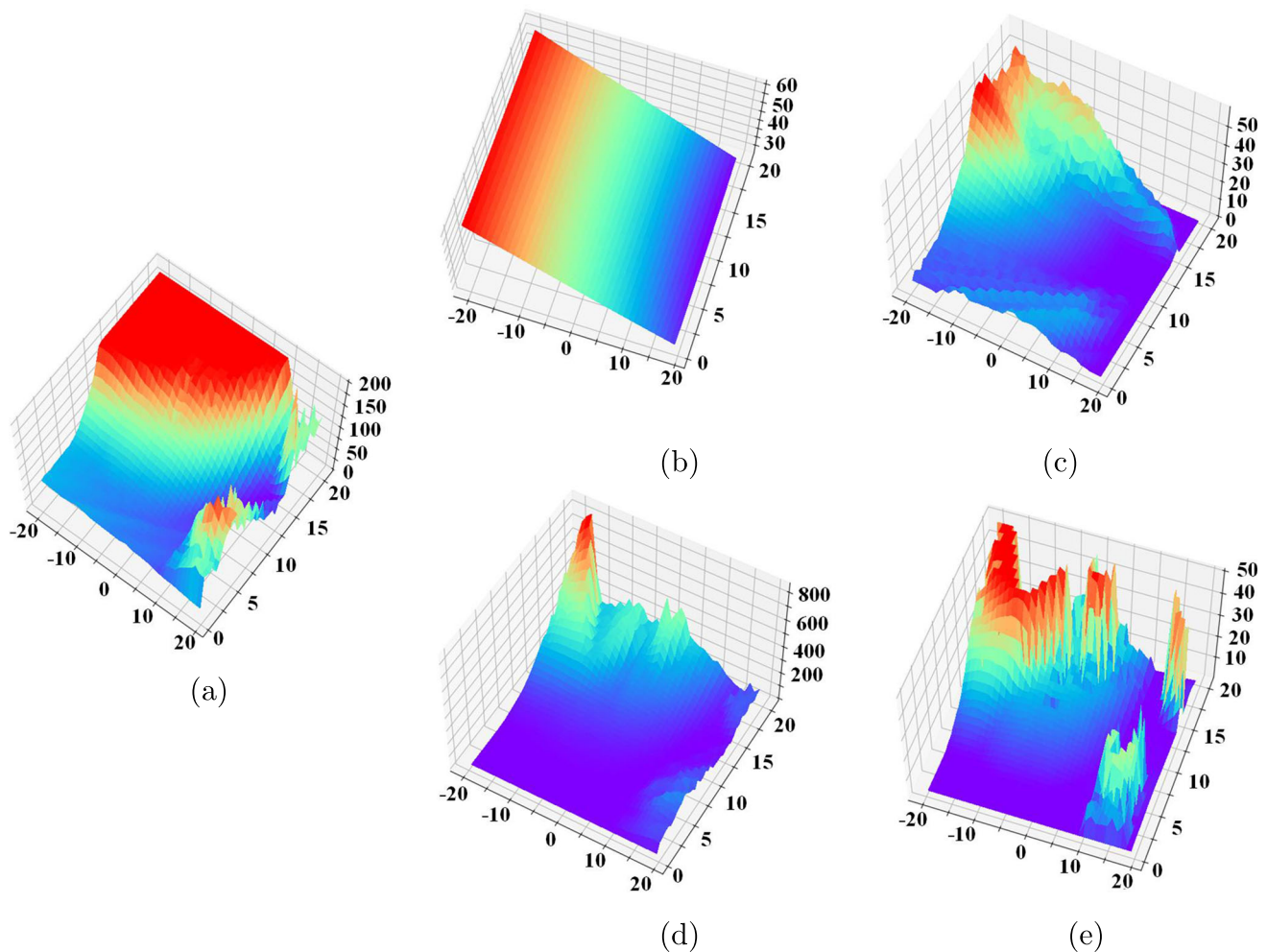


Fig. 6 (a) the overall evaluation map corresponding to (1), (b), (c), (d), (e) are the ingredients of (a) which correspond to (2), (3), (4) and (5) respectively

or (d). If the average reward or success rate surpasses the previous test results, the model is updated. Thus, by the end of training, each hyperparameter set corresponds to a unique network model, from which the optimal one is selected. A series of results, subjected to the selected hyperparameters, in terms of the variation of SR, SPL, SPVR, TS and GHD for the training stage, are shown in Fig. 8.

When the learning rate is set stable and set to decrease linearly separately, it is observed that the model with a linearly decreasing learning rate performs better. This improvement is mainly because, in the later stages of training, the model stabilizes. If the learning rate remains high, as in the early stages, the model may become unstable. The update epoch refers to the number of times the same batch of data is repeatedly optimized. A smaller update epoch can result in low data utilization efficiency and longer learning cycles. Conversely, a larger update epoch allows the policy to learn more deeply from the same batch of data, but it may also lead to overfitting. Therefore, it is crucial to choose an appro-

priate update epoch. The same principle applies to update steps. Fewer update steps result in lower computational costs and shorter training times but may fail to fully capture the environment's dynamics, leading to unstable or underfitting policies. More update steps can provide a richer set of sample data, improving the estimation of the policy and value function. However, this may increase computational costs and training times. Referring to Fig. 8, the hyperparameters with a combination of linear-10-3072 (linear learning rate-number of update epochs-update step size) can be selected as optimal.

4.4.2 Model validation based on uncertainty quantification

To ensure the stability and optimality of the selected model, uncertainty is estimated using a novel ensemble-based automated randomly deactivated weight database (ARDCW) approach [42]. The ARDCW focuses solely on randomly deactivating weights, not neurons, compelling the remaining

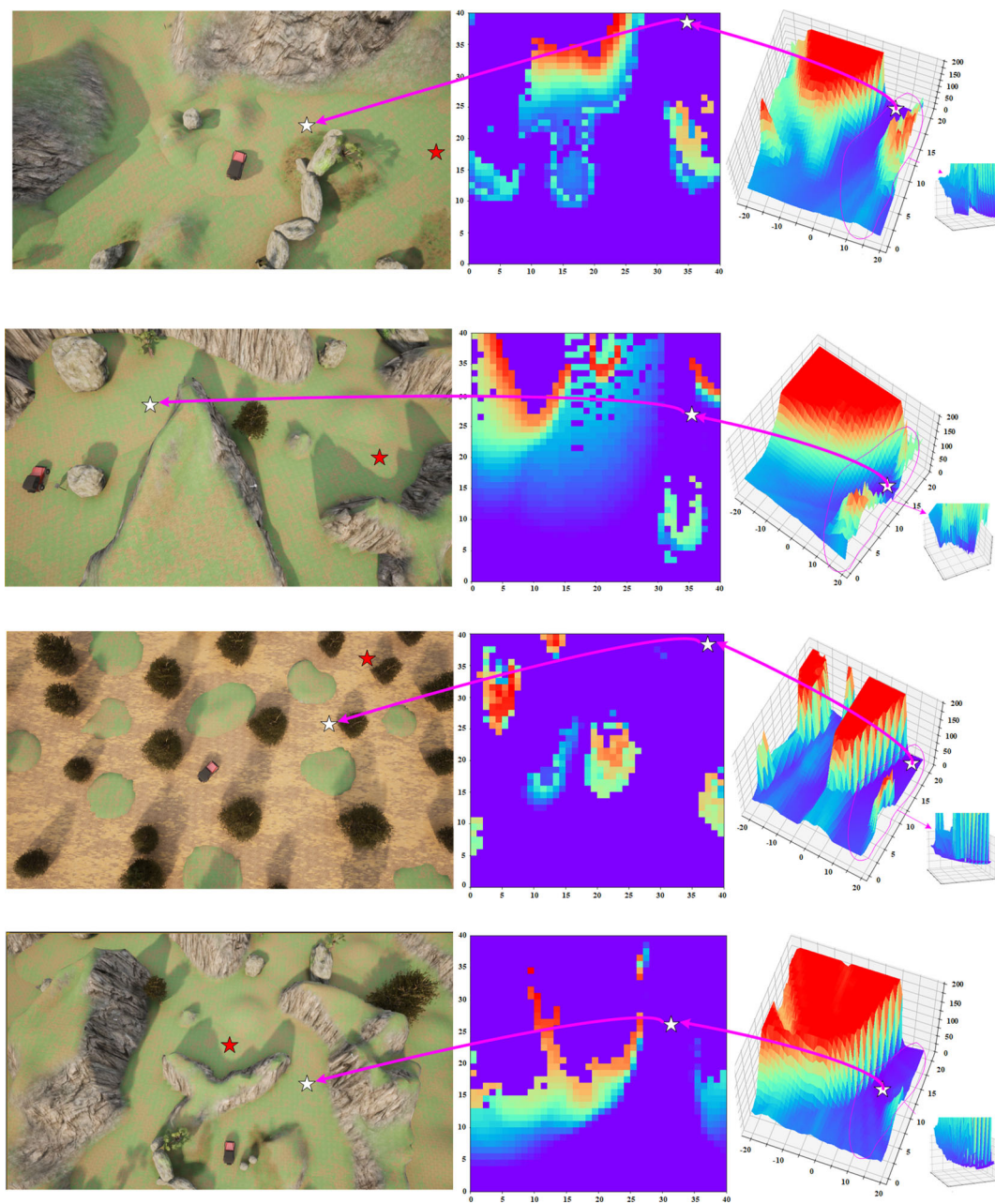


Fig. 7 Four typical positions and their corresponding cost map, evaluation map and best wayshower

weights to participate more actively in the learning process and aiding in the reduction of overfitting. This stage involves conducting experiments on two new maps (Fig. 5 (e) and (f)), which differ from the four training maps in terms of obstacle distribution. Once the optimal model has been identified, the weights associated with the processing elements are saved in a database as a .pth file, allowing for direct application to validation or untrained maps for subsequent driving tests. In Block B, only the finalized weight database, resulting from hyperparameters optimization stage, is utilized. The ARDCW approach is applied specifically to the

weight database of the optimal model. The model is then re-evaluated in validation maps with selected weight components deactivated, facilitating the monitoring of variations in driving performance across different scenarios. In [42], The minimum number of sampled weights is 1, while the maximum number of desired droppings was set at 50% of the whole connective weights. In this study, considering the increased count of connective weights compared to [42], the number of the deactivated weight is chosen from 5% to 50%, every 5% interval. Each interval is tested ten times (without repetition, even for the same deactivated weight percentage,

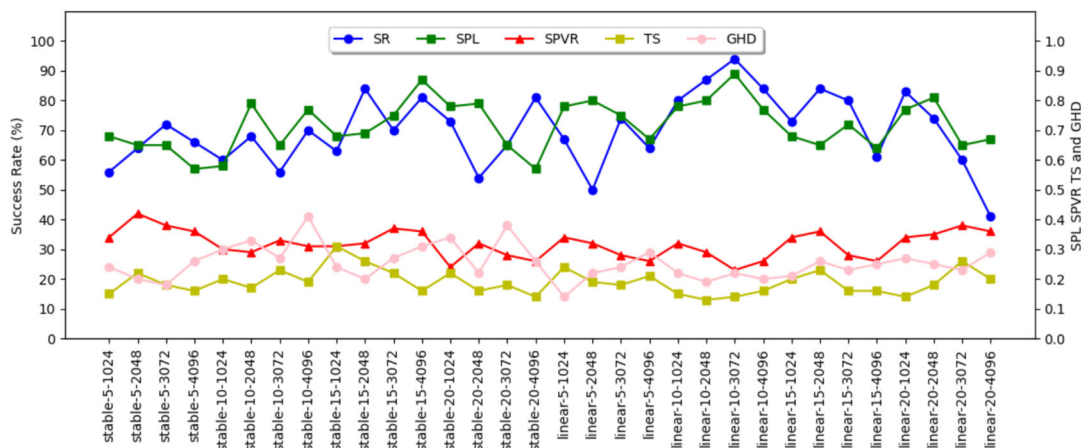


Fig. 8 Identifying the optimum model through variation of SR, SPL, SPVR, TS and GHD

the model differs due to the randomized selection of deactivated weights), resulting in a total of 100 trials. This thorough testing allows for the identification of the optimal model with the best weight configuration by comparing the outcomes of these experiments.

Figure 9 shows the results planned by the optimal model with optimal weight on the two validation maps, where ten start and goal location pairs were chosen to test the model. The model successfully planned all the paths, and this experiment was repeated twenty times, resulting in a total of 200 trials, with an average success rate of 90%. These results demonstrate that after model validation and uncertainty quantification, the model can maintain good performance when faced with new driving scenarios.

5 Performance comparison and discussion

5.1 Comparison of learning speed

Figure 10 illustrates the learning curves of the proposed method and the other DRL baselines, with each of the four figures corresponding to the training results on four maps in Carla, respectively. The total learning timesteps are

1,000,000, and the learning curves depict the cumulative reward per episode averaged over every 2000 timesteps.

For the first map, all DRL-based baselines were successfully trained, and their models converged. It is challenging to discern which one converges the fastest due to the simplicity of the map and navigation task. In the second and third maps, it is evident that LMAW converges faster than the other four baselines. The cumulative reward increases rapidly at the beginning of training as the network quickly learns from the positive rewards obtained by following the guidance of wayshowers. This guidance simultaneously helps steer the UGV away from obstacles, thereby enhancing the efficiency of state space exploration. In contrast, the two goal-driven baselines explore the environment less efficiently, as they receive sparse rewards related only to the goal. Despite being guided by sub-goals during training, Terp and IDLE still exhibit lower exploration efficiency compared to LMAW but higher than the other two goal-driven approaches. For the fourth map, only the proposed network, IDLE and Terp converge, while the other baselines fail. In this map, numerous long obstacles obstruct the UGV's path. When the goal lies on the opposite side of an obstacle, to approach the goal, the UGV must initially drive towards it. However, to ensure a collision-free path, the UGV should veer away from the goal,

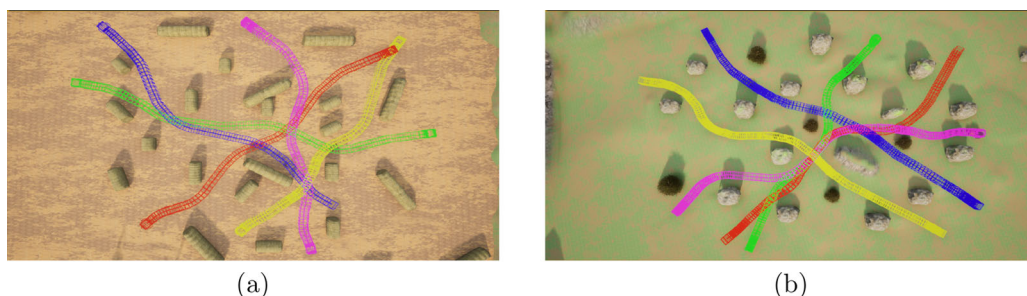


Fig. 9 Validation results planned by LMAW in validation maps

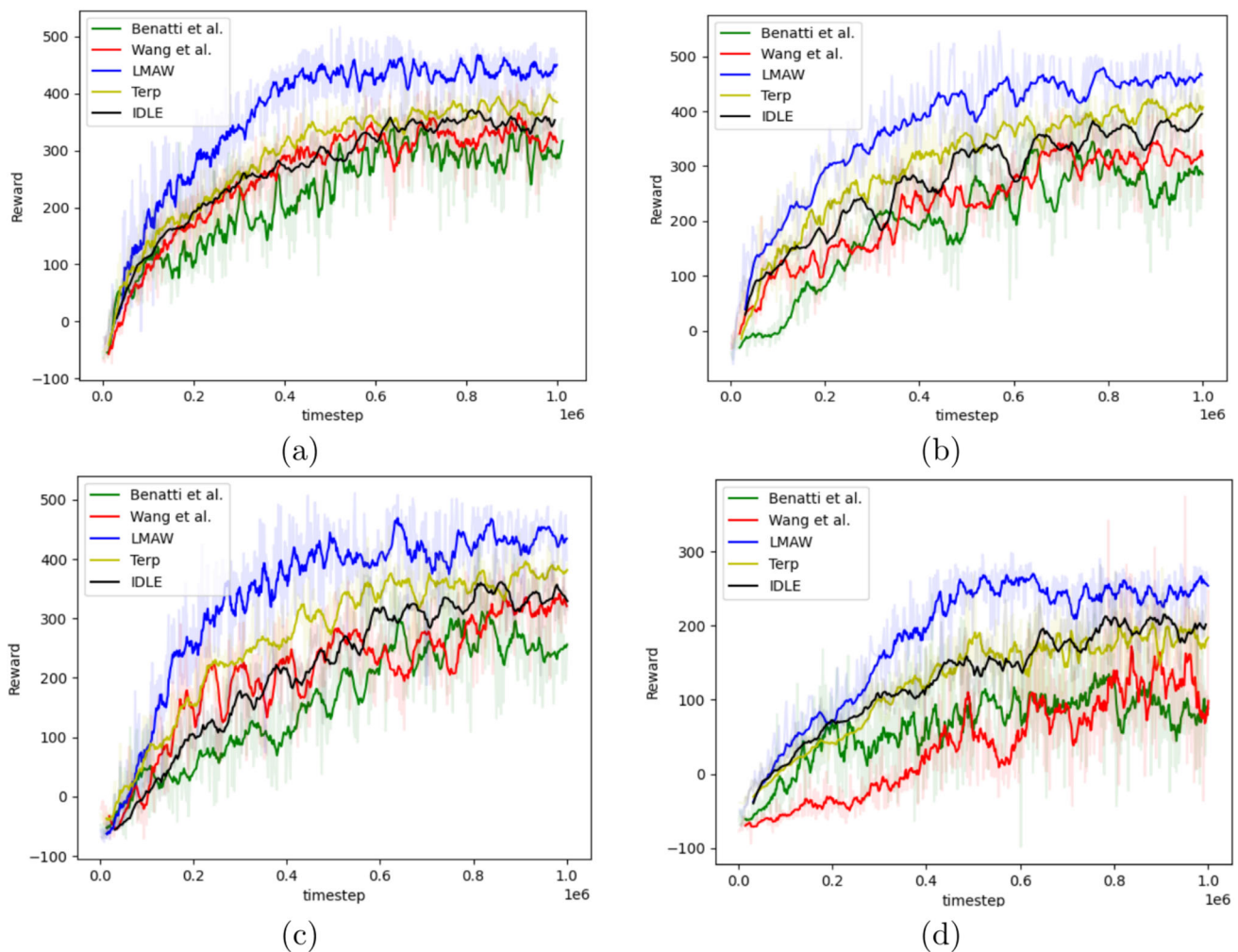


Fig. 10 Learning curves of the training in the four maps of Carla

resulting in a substantial difference in heading angles. Neither of these actions is advantageous for the goal-driven baselines, as the sparse rewards provided do not incentivize such actions. Conversely, the proposed heuristic function selects wayshowers to guide the UGV, allowing it to continuously receive positive and dense rewards. Consequently, the UGV can navigate around the long obstacles under the guidance of wayshowers and successfully reach the goal.

5.2 Comparison of visualized paths and driving speed variation

To compare with traditional approaches [37, 38], the experiment was conducted in the Gazebo simulation environment. This decision was made because the two traditional algorithms were designed for specific dynamics models, potentially leading to failure if directly applied in Carla. To demonstrate the ability to navigate through cluttered environments, two maps were designed (see Fig. 11). The second

map was derived from the first map and featured dense obstacles. The testing results, summarized in Table 1, showcase the performance of each method for various start and end points. The first line in each pair represents the path length in the first map, while the second line denotes the path length in the second, more cluttered map. Since traditional methods are not specifically designed for stable velocity planning, SPVR was not compared in this experiment while the proposed approach addresses this aspect. For Far, FASTER, and LMAW, the success rate (SR) was consistently 100%. Regarding success weighted by path length (SPL), the values were 0.91, 0.88, and 0.95, respectively. In terms of path length, SR, and SPL, LMAW demonstrated superior performance. Furthermore, LMAW offers versatility, being applicable in both Carla and Gazebo simulations with minimal modifications. In contrast, Far and FASTER require significant expert input for parameter tuning.

For the verification experiment of DRL approaches, five start and goal location pairs were selected in each of the four

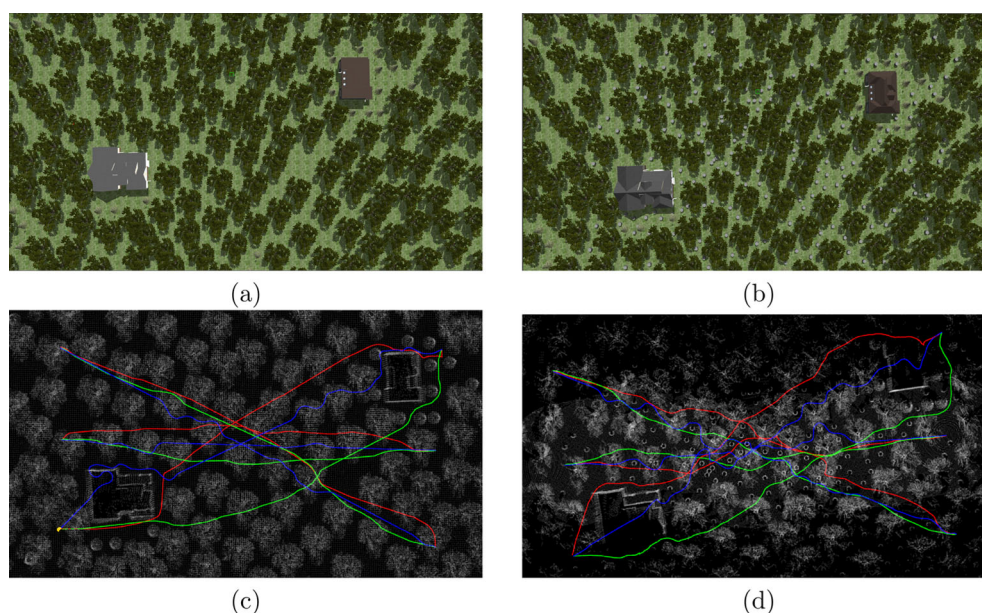


Fig. 11 Maps in gazebo simulation and planning result. Paths colored red, blue and green are planned by Far, FASTER and LMAW respectively

maps, resulting in a total of twenty paths. These pairs were predefined for the UGV to navigate. Each trial was deemed successful only if the UGV autonomously reached the goal point within a position tolerance of 1.2 meters and within a maximum of 1000 timesteps. All paths planned by each algorithm are visualized in Fig. 12, with paths colored yellow, green, red, blue, and black representing Terp, Benatti et al., Wang et al., LMAW, and IDLE respectively. Path length is used as the evaluation metric, and the results are recorded in Table 2. The bold values indicate the shortest length for each route, while the symbol ‘\’ denotes a planning failure for that route. In the first three maps, the four other DRL-based baselines could plan the routes, as their models were well-trained and converged. However, they either failed or produced suboptimal planning results in the fourth map due to non-ideal training outcomes. Only LMAW successfully planned all the routes. Among the twenty paths, thirteen planned by LMAW were the shortest, demonstrating that our method outperforms the others in terms of path length and convergence speed.

Besides path length and convergence speed, driving stability is also crucial for path planning. This can be evaluated by assessing whether the UGV maintains a predefined velocity interval. The speed curves of all the routes planned by the four algorithms are visualized in Fig. 12. For the other four DRL-based baselines, the driving policy exhibits significant variability across the four maps, sometimes even reaching very high speeds. In contrast, the UGV controlled by the proposed method demonstrates the most stable driving behavior, maintaining a consistent speed. This indicates that the proposed method excels in driving stability compared to the other methods.

Furthermore, twenty start and goal location pairs were chosen in the fourth map to test the model. The LMAW method successfully planned all the paths (see Fig. 13). This experiment was repeated ten times, resulting in a total of 200 trials, with an average success rate of 90%. Speed curves for the twenty routes are shown in Fig. 14. The two red transverse lines in figure represent the lower and upper limits of the suitable speed range. The figure indicates that the UGV

Table 1 Length of testing routes in the two maps of gazebo

Start(x, y, z) End(x, y, z)	Far [37]	FASTER [38]	Ours
[−42.0, −44.0, 0.7], [10.0, 48.0, 0.2]	108.7	109.5	107.5
	110.8	117.9	108.2
[7.0, −47.0, 0.3], [−35.0, 49.0, 0.2]	116.7	122.5	115.4
	121.4	132.9	117.3
[−13.0, 48.0, 0.4], [−18, −45, 0.2]	95.1	95.3	94.2
	97.9	102.4	95.3

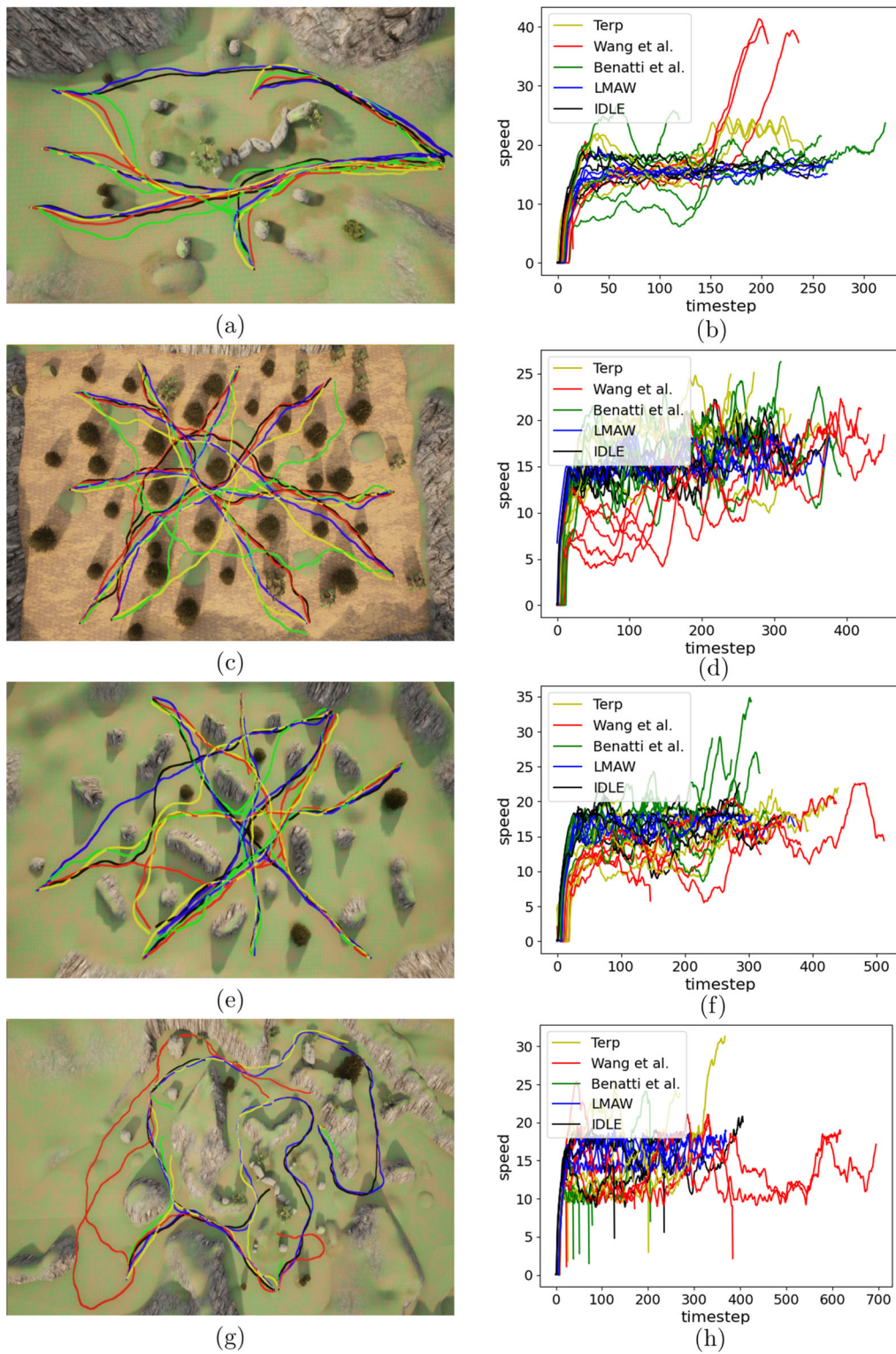


Fig. 12 Testing results in the four maps. Paths and speeding curves colored yellow, green, red, blue and black are planned by Terp, methods proposed by Benatti *et al.*, Wang *et al.*, LMAW and IDLE respectively. The path diagram on the left and the speed curve on the right correspond one to one

Table 2 Length of testing routes in the four maps of Carla

Start(x, y) End(x, y)	Terp [21]	Wang [40]	Benatti [39]	IDLE [20]	Ours
[3.17, 26.49], [72, 32.5]	70.49	70.5	74.8	70.6	69.9
[4.18, 17.13], [72, 32.5]	73.78	74.8	78.7	73.1	74.6
[2.05, 35.14], [72, 32.5]	70.37	70.9	71.6	70.5	69.8
[39.31, 20.7], [72, 32.5]	40.76	37.8	38.4	37.3	37.9
[37.9, 45.6], [72, 32.5]	45.25	42.2	49.3	46.9	45.9
[−80.1, 41.9], [−25.7, 112]	90.5	94.0	101.7	95.15	89.7
[−26.1, 37.7], [−75.7, 123]	100.92	102.7	113.2	103.74	99.8
[−18.6, 56.8], [−84.7, 100]	81.6	81.9	84.6	83.01	79.5
[−80.0, 49.8], [−20.8, 86.5]	77.48	73.6	73.8	73.81	69.3
[−50.4, 38.3], [−55.7, 126]	89.21	93.4	96.2	92.46	89.5
[−42, −23.9], [−87.8, −109]	101.8	116.8	105.6	108.89	101.6
[−71.8, −130], [−31.6, −56.8]	\	105.9	89.3	85.63	86.1
[−24.2, −114], [−92.4, −54]	92.8	95.5	97.5	91.8	93.9
[−93.0, −82.5], [−24.7, −82.9]	70.2	\	71.6	70.48	69.8
[−25.6, −53.7], [−87, −111]	88.4	86.7	86.2	87.05	85.9
[60.5, 46.8], [93.1, 61.9]	\	\	\	97.4	99.5
[16.7, −23.1], [71.18, 29.93]	\	156.2	\	\	104.5
[77.3, −18.5], [11.0, 24.9]	87.4	162.3	\	85.4	85.2
[11.0, 24.9], [71.2, 29.9]	75.05	\	\	85.0	86.1
[77.3, −18.5], [93.1, 65.9]	100.58	\	\	\	99.9

drives at a suitable speed, ranging from 14 to 18 km per hour, for most of the time. This strongly demonstrates the stable driving policy of the proposed model.

5.3 Comparison of metric evaluation

It is challenging to evaluate the reliability of a model using only one evaluation criterion. For instance, SR and SPL reflect the probability of successful path planning and the path length, respectively, but do not quantitatively assess the quality of the planned path. Similarly, SPVR indicates the stability of the driving strategy but does not account for the

TS of the path. GHD reflects the degree of deviation of the UGV heading from the endpoint but also fails to consider the TS of the planned path. In this section, five evaluation criteria are used to comprehensively assess the quality of the model. The driving performance results, in terms of SR, SPVR, SPL, TS, and GHD, are presented in Table 3. Accordingly, a model with lower SPVR, TS, and GHD, and higher values of SR and SPL, indicates better performance.

Goal-guided methods [39, 40] typically define the reward function based on the direct Euclidean distance from the current UGV position to the final goal. This straightforward approach incentivizes the UGV to minimize the distance to the goal in a straight line. The simplicity of this method makes it easy to implement and understand, as it directly focuses on achieving the ultimate objective. However, relying solely on a direct path can be problematic in environments with unpredictable obstacle distributions. Experimental results have validated that the SR and SPL during validation are significantly inferior compared to training. When applied to cluttered maps or new environments, planning failures are frequent, and model generalization is poor. Moreover, it is worth noting that despite the reward function being directly defined on the goal point, the GHD for the two goal-guided methods is larger than that for the sub-goal guided approaches and LMAW. This suggests that the model has not fully comprehended the navigation task. Additionally, higher SPVR and TS values reveal the instability of the model, as

**Fig. 13** Routes planned by LMAW in the fourth map

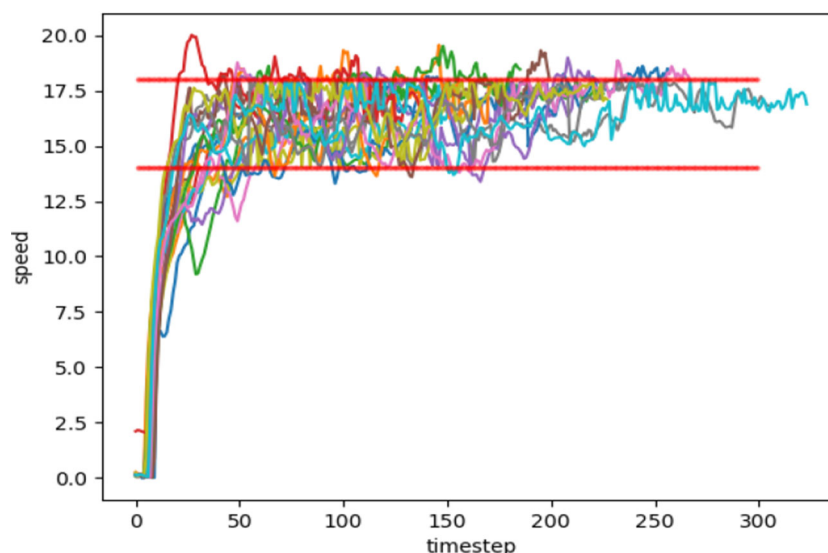


Fig. 14 Speed curves of the 20 routes in the fourth map

path smoothness and speed variation are the best indicators of a stable driving strategy.

Sub-goal guided approaches [20, 21] decompose the navigation task into sequential sub-goals or waypoints distributed along the path to the final destination. These waypoints are strategically selected to align with the anticipated path toward the goal, allowing the UGV to navigate through a series of intermediate objectives until reaching the ultimate target. As supported by the results from experimental results that the SR, SPL and TS of validation are not much different from the training, this segmentation strategy enhances robustness against variations in obstacle distributions. However, while sub-goal guided methods reduce the complexity

of the navigation task, they primarily rely on the final goal point for waypoint selection. Once the UGV begins navigating towards a waypoint, the information regarding the final goal point is no longer utilized, leading to higher GHD than LMAW. This simplification can lead to suboptimal planning outcomes, often resulting in lower SPL to the proposed method.

The proposed model, as demonstrated by experimental results, shows superior performance compared to both goal-guided and sub-goal guided approaches. The key innovation lies in the integration of wayshower information, which significantly enhances the network's decision-making process. This integration allows for more adaptive navigation behavior, enabling the network to dynamically adjust its trajectory based on real-time observations rather than rigidly adhering to predetermined waypoints. As a result, the model achieves a higher SR and SPL in navigating the UGV toward its goal and exhibits better generalization capabilities. Furthermore, the awareness of both the intermediate wayshower and the final goal point throughout the navigation process reduces GHD and improves TS. This holistic approach to navigation ensures smoother trajectories and more stable driving strategies, leading to overall superior performance in various environments.

5.4 Uncertainty estimation

The ARDCW [42] method is utilized for estimating uncertainty, which helps in evaluating the confidence of each method's outputs. The core feature of ARDCW lies in the random deactivation of connective weights within the neural network. ARDCW can be interpreted as an ensemble-based Bayesian approximation of the Gaussian Process probabilis-

Table 3 Criteria to evaluate and compare the employed models

		SR	SPVR	SPL	TS	GHD
Terp [21]	Train	83	0.35	0.75	0.17	0.28
	Test	80	0.37	0.68	0.21	0.27
	Validation	75	0.40	0.70	0.20	0.25
Wang [40]	Train	76	0.38	0.67	0.26	0.30
	Test	70	0.40	0.58	0.30	0.36
	Validation	45	0.44	0.55	0.34	0.37
IDLE [20]	Train	87	0.31	0.84	0.17	0.26
	Test	80	0.35	0.74	0.19	0.26
	Validation	82	0.34	0.77	0.22	0.24
Benatti [39]	Train	74	0.37	0.69	0.30	0.28
	Test	65	0.42	0.61	0.34	0.30
	Validation	48	0.40	0.50	0.32	0.27
Ours	Train	93	0.24	0.88	0.19	0.21
	Test	88	0.27	0.82	0.22	0.20
	Validation	90	0.26	0.84	0.20	0.22

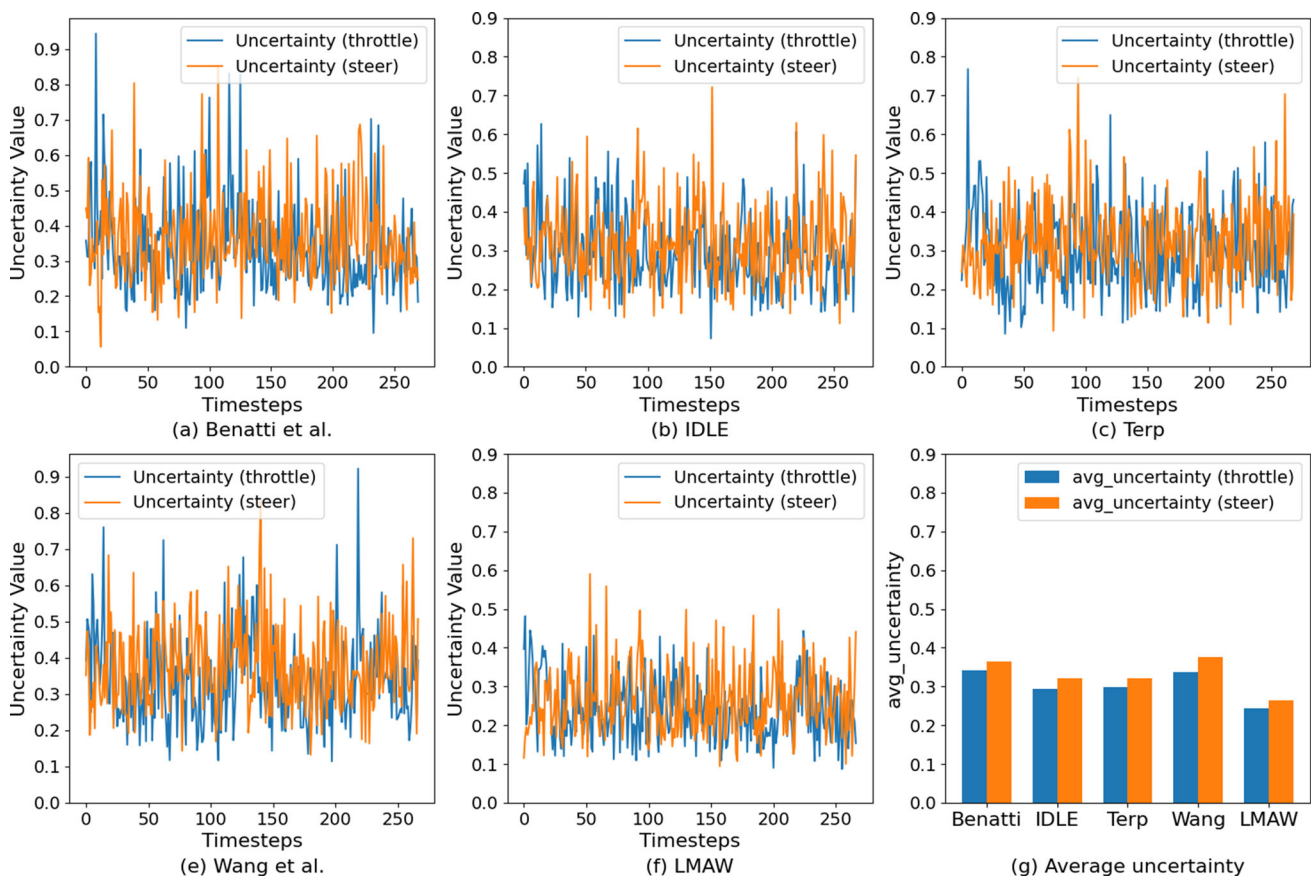


Fig. 15 Result of uncertainty estimation using ARDCW

tic model. The most significant advantage of ARDCW is that it does not require any modifications to the existing network architecture. This method can be directly applied to models that have already been trained, making it a practical choice for uncertainty quantification.

One of the uncertainty estimation results is illustrated in Fig. 15. In the first five subplots, at timestep 0, the UGV is positioned at the starting point, and by the final timestep, it has reached the endpoint. For each timestep and state transition, the model's output is computed ten times. During each computation, connective weights are randomly deactivated, and the output is calculated. The mean of these ten outputs is used as the final control command applied to the UGV for transitioning to the next state. The outputs from each

computation are collected, and their variance is calculated. This variance quantifies the model's uncertainty regarding its predictions, providing insights into the reliability of the model's outputs. In the plots, the yellow and blue lines represent the uncertainty in the policy outputs—specifically, throttle and steering—for each evaluated DRL approach. These lines illustrate how uncertainty varies across different methods. In the final subplot, the yellow and blue bars depict the averaged uncertainty for each approach, summarizing the overall levels of uncertainty. The results indicate that the proposed model exhibits higher confidence in its outputs compared to the baseline methods. Specifically, it demonstrates lower uncertainty, which suggests that the model's predictions are more reliable and consistent. This enhanced confidence in the

Table 4 Estimation of aleatoric uncertainty in control noise

Noise	LMAW		Terp [23]		Wang [40]		IDLE [20]		Benatti [39]	
	N	10%	N	10%	N	10%	N	10%	N	10%
SR	93	90	83	75	76	68	87	82	74	67
SPVR	0.24	0.26	0.35	0.38	0.37	0.44	0.31	0.34	0.37	0.41
SPL	0.88	0.84	0.75	0.70	0.67	0.55	0.84	0.77	0.69	0.50
TS	0.19	0.22	0.17	0.20	0.26	0.31	0.17	0.19	0.30	0.35
GHD	0.21	0.23	0.28	0.32	0.30	0.34	0.26	0.29	0.28	0.34

proposed model underscores its effectiveness and stability in managing complex tasks.

Furthermore, the aleatoric uncertainty of all obtained strategies is evaluated by conducting experiments with additional noise (10% of the control domain) added to the output of the strategy. This noise simulates real-world conditions where navigation tasks can be affected by various sources of unknown compound noises. Gaussian noise is used due to its ability to realistically represent such complex noise scenarios. The metrics SR, SPL, SPVR, TS, and GHD under noise are compared to those without noise, with the results shown in Table 4. The results indicate that DRL-based strategies achieve favorable robustness in autonomous driving tasks. This anti-noise capability can be attributed to the robust learning characteristics of the RL algorithm, specifically the PPO algorithm, rather than being directly linked to the environment model. Nonetheless, the proposed strategy demonstrates good robustness, indicating its significant potential for applications in autonomous driving.

5.5 Sensitivity analysis

To confirm the impact of each task state on model performance, a series of eight carefully controlled experiments were conducted. In each of these experiments, a specific task state was systematically removed to observe its effect on the model's overall performance. The training outcomes were evaluated using several key metrics: SPVR, SR, SPL, TS, and GHD, as detailed in Table 5. It is important to note that the symbol 'N' indicates instances where the training process failed entirely, underscoring the significance of the missing task state in those cases.

When either *throttle* or *speed* was removed, the model experienced almost no degradation in performance. This outcome is likely due to the strong correlation between these two variables. Since acceleration can be calculated from speed over time, and speed can be derived from the integral of acceleration, the model can effectively compensate for the absence of one by relying on the other. This redundancy explains why

Table 5 Result of sensitivity experiments for task states based on evaluation metrics

		SR	SPVR	SPL	TS	GHD
Full task inputs (base case)	Train	93	0.24	0.88	0.19	0.21
	Test	88	0.27	0.82	0.22	0.20
	Validation	90	0.26	0.84	0.20	0.22
steer excluded	Train	92	0.23	0.79	0.26	0.28
	Test	89	0.28	0.74	0.30	0.32
	Validation	88	0.25	0.75	0.28	0.31
throttle excluded	Train	94	0.23	0.87	0.20	0.22
	Test	90	0.25	0.82	0.23	0.20
	Validation	90	0.26	0.85	0.21	0.22
speed excluded	Train	92	0.24	0.87	0.18	0.23
	Test	87	0.26	0.84	0.24	0.25
	Validation	90	0.26	0.83	0.22	0.23
θ_{roll} excluded	Train	80	0.27	0.65	0.35	0.31
	Test	75	0.32	0.62	0.39	0.35
	Validation	73	0.30	0.61	0.37	0.33
θ_{pitch} excluded	Train	83	0.24	0.78	0.33	0.31
	Test	77	0.29	0.72	0.37	0.30
	Validation	72	0.27	0.70	0.34	0.32
d_{goal} excluded	Train	92	0.24	0.86	0.22	0.23
	Test	89	0.27	0.82	0.25	0.20
	Validation	88	0.26	0.83	0.23	0.22
α_{goal} excluded	Train	N	N	N	N	N
	Test	N	N	N	N	N
	Validation	N	N	N	N	N
$\alpha_{wayshower}$ excluded	Train	N	N	N	N	N
	Test	N	N	N	N	N
	Validation	N	N	N	N	N

the removal of either variable does not significantly hinder the model's ability to perform.

In contrast, removing the *steer* task state resulted in a noticeable decline in performance, particularly in the SPL, TS, and GHD metrics. This can be attributed to the model's need for consistent historical steering data to make accurate decisions. Without knowledge of previous steering actions, the model's steering decisions become erratic between frames, leading to a substantial drop in performance.

The removal of the orientation-related parameters θ_{roll} and θ_{pitch} also had a detrimental impact on the model's performance. These parameters are crucial, especially in unstructured environments, as they provide the UGV with critical information about its current posture. Without this information, the model struggles to make informed decisions, leading to suboptimal performance.

On the other hand, the removal of α_{goal} and $\alpha_{wayshower}$ had an even more severe effect: the model failed to converge during training. This failure highlights the indispensable role these two parameters play in the model's decision-making process. The goal and wayshower information are evidently crucial for the UGV's ability to navigate effectively, and their absence renders the model incapable of learning.

Interestingly, the removal of d_{goal} -which represents the distance to the goal-did not significantly affect the model's performance. This is likely because the information about the distance to the goal is already embedded in the selection of the wayshower, making d_{goal} somewhat redundant in the presence of accurate wayshower data.

These findings collectively emphasize that the UGV's navigation and decision-making processes are heavily reliant on goal and wayshower information, as these parameters are fundamental to the model's ability to function effectively. The results also underscore the importance of maintaining a comprehensive set of task states, as the removal of critical parameters can severely compromise the model's performance.

5.6 Ablation study

The results of the previous section show that the proposed LMAW can effectively improve the learning speed and final performance. In this section, further study is conducted to explore the impact of the parameter η , wayshower and length attention mechanism on learning performance.

The parameter η is crucial to the final performance as it determines the impact of wayshowers and the goal point on navigation during training. To explore the impact of the η parameter on learning performance, five different values of η are compared. Figure 16 illustrates the learning curves when η ranges from 0.1 to 0.9. The value of the η parameter influences learning performance, with variations in final performance and early improvement rates. Regardless of the

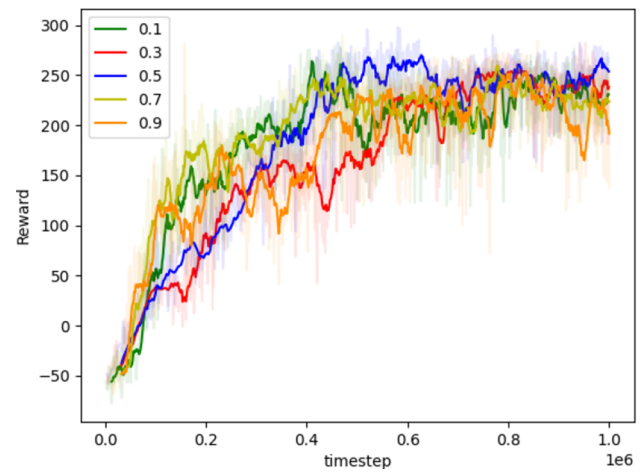


Fig. 16 Learning curves of different η

η value, the model ultimately converges. For smaller η values, the goal point's guidance is less prominent, resulting in slower performance improvement. For larger η values, the performance improvement speed is also reduced due to excessive emphasis on the goal point. It should be noted that the total reward increases with larger η values because the reward components r_d and r_h change with η . Even though the learning curves for $\eta=0.5$ and $\eta=0.7$ appear similar, the trained models' performance varies significantly. Therefore, a moderate value (0.5) for η was chosen, which ensures faster learning speed and a more stable driving policy.

To demonstrate the impact of wayshowers and length attention mechanism on learning performance, LMAW, LMAW without the guidance of wayshowers (LMA) and LMAW without both the guidance of wayshowers and the length mask mechanism (CT) are compared, the models are retrained and the learning curves are shown in the Fig. 17. The learning curve for LMA is superior to CT, indicating that the proposed length attention mechanism provides ben-

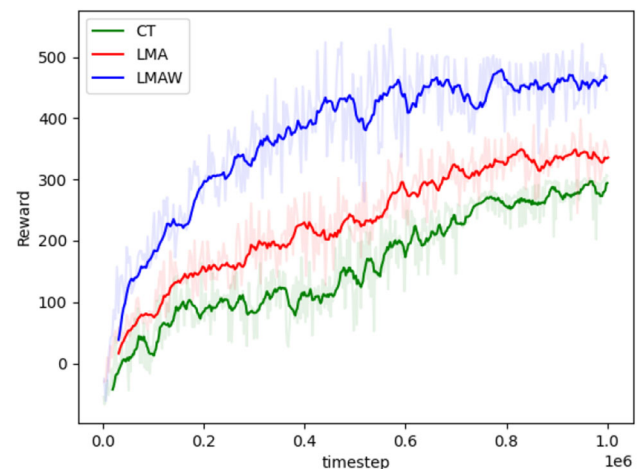


Fig. 17 Learning curves of LMA, CT and LMAW

efit during training. Moreover, the learning curve for LMAW converges more rapidly than those for LMA and CT, demonstrating the significant performance improvement attributed to the guidance of wayshowers. Since both CT and LMA are goal-guided methods, they exhibit the limitations discussed previously. However, it is worth noting that LMA converges faster than CT, denoting the training efficiency brought by length mask mechanism.

5.7 Discussion

Despite significant improvements made above, it is acknowledged that there are still areas where further improvement is possible. The classic actor-critic (AC) architecture, which underpins many DRL autonomous driving frameworks based on PPO, is limited because the policy does not directly participate in the value function update. This can lead to inaccurate value function estimation, thereby affecting the performance of the PPO algorithm. Introducing policy feedback into the update process of the value function could be a potential direction to mitigate value overestimation [43]. Another drawback of PPO is its inferior sample efficiency compared to off-policy algorithms, such as Soft Actor-Critic (SAC), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Deep Deterministic Policy Gradient (DDPG). These algorithms have shown substantial progress, with innovations including the use of ensembles of Q functions with Dropout [44], the application of batch normalization and the elimination of target networks [45]. Furthermore, learning embeddings that capture intricate state-action interactions [46], and extending relative entropy regularization to the AC framework, have also contributed to their success. While these methods have excelled in simple control tasks, their application in autonomous driving scenarios and subsequent refinement based on empirical results represent promising research directions. Applying these algorithms to autonomous driving offers numerous benefits, including but not limited to reducing network complexity, shortening training times, and enhancing overall network performance. Moreover, it can catalyze further advancements in algorithms specifically designed to tackle the unique challenges posed by dynamic and unpredictable driving scenarios.

Additionally, enhancing the sim-to-real transferability of the model is crucial. Directly applying the trained model to UGVs in real-world scenarios presents numerous challenges. Unlike the ideal sensor data in simulation, sensors in the real world are noisy and affected by various elements [47, 48], thus additional considerations are necessary for real-world applications. The following points can serve as extensions for future work: 1) Develop a denoising network that incorporates historical frame data to mitigate outliers caused by varying weather conditions and other uncertainties. 2) Create

a point cloud database of real-world scenarios to pre-train the denoising network. This pre-trained model can then be integrated into the reinforcement learning framework to enhance the model's sim-to-real capabilities. Human guidance has emerged as another powerful tool to boost the efficiency and performance of reinforcement learning algorithms in sim-to-real applications [28, 49, 50]. Techniques such as human-guided learning objectives, prioritized human experience replay, and human intervention-based reward shaping have been shown to improve the effectiveness of human guidance. By incorporating these mechanisms, humans can intervene and correct the agent's unreasonable actions in real time during the model training process, thereby improving the model's performance in real-world settings.

6 Conclusion

In this article, a novel deep reinforcement learning approach is presented for mapless autonomous navigation with the guidance of wayshowers. Specifically, a new metric based on multilevel analysis is developed to generate elevation maps, aiding in the identification of optimal wayshowers. By integrating wayshower information with other inputs, a MHA module is incorporated into the DRL network, which includes a length attention mechanism to enhance focus on recent historical observation sequences and promote model convergence. Additionally, the reward function is reshaped to provide dense reward signals, effectively addressing the sparse reward problem inherent in goal-driven methods and improving navigation in challenging environments. Experiments on hyperparameter optimization and model validation were conducted to obtain the optimal model with the best-performing weights. Comparative evaluations using several metrics demonstrate that the proposed approach outperforms existing methods, achieving higher SR and SPL, as well as lower SPVR, TS and GHD. The experimental results also indicate greater generalization ability when the model is applied to new scenarios, suggesting its potential for real-world applications.

Acknowledgements This work is co-funded by the National Science and Technology Innovation 2030 of China-New Generation Artificial Intelligence (Grant number: 2022ZD0115603), the Key R&D program of Jiangsu Province (Grant number: BE2022053-5).

Data Availability Data sharing is not applicable to this article as no new data were created or analyzed in this study.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

References

- Kiran BR, Sobh I, Talpaert V et al (2021) Deep reinforcement learning for autonomous driving: A survey. *IEEE Trans Intell Transp Syst* 23(6):4909–4926. <https://doi.org/10.1109/TITS.2021.3054625>
- Thoresen M, Nielsen NH, Mathiassen K et al (2021) Path planning for uavs based on traversability hybrid a. *IEEE Robot Autom Lett* 6(2):1216–1223. <https://doi.org/10.1109/LRA.2021.3056028>
- Qi Y, He B, Wang R et al (2022) Hierarchical motion planning for autonomous vehicles in unstructured dynamic environments. *IEEE Robot Autom Lett* 8(2):496–503. <https://doi.org/10.1109/LRA.2022.3228159>
- Ye L, Wu F, Zou X et al (2023) Path planning for mobile robots in unstructured orchard environments: An improved kinematically constrained bi-directional rrt approach. *Comput Electron Agric* 215:108453. <https://doi.org/10.1016/j.compag.2023.108453>
- Roriz R, Cabral J, Gomes T (2022) Automotive lidar technology: A survey. *IEEE Trans Intell Transp Syst* 23(7):6282–6297. <https://doi.org/10.1109/TITS.2021.3086804>
- Kong D, Li X, Xu Q et al (2024) Sc_lpr: Semantically consistent lidar place recognition based on chained cascade network in long-term dynamic environments. *IEEE Trans Image Process* 33(1):2145–2157. <https://doi.org/10.1109/TIP.2024.3364511>
- Chib PS, Singh P (2023) Recent advancements in end-to-end autonomous driving using deep learning: A survey. *IEEE Trans Intell Veh* 9(1):103–118. <https://doi.org/10.1109/TIV.2023.3318070>
- Li F, Guo C, Luo B et al (2021) Multi goals and multi scenes visual mapless navigation in indoor using meta-learning and scene priors. *Neurocomputing* 449:368–377. <https://doi.org/10.1016/j.neucom.2021.03.084>
- Jiang H, Esfahani MA, Wu K et al (2022) itd3-cln: Learn to navigate in dynamic scene through deep reinforcement learning. *Neurocomputing* 503:118–128. <https://doi.org/10.1016/j.neucom.2022.06.102>
- Staroverov A, Yudin DA, Belkin I et al (2020) Real-time object navigation with deep neural networks and hierarchical reinforcement learning. *IEEE Access* 8:195608–195621. <https://doi.org/10.1109/ACCESS.2020.3034524>
- Li Z, Zhou A (2023) Rddrl: a recurrent deduction deep reinforcement learning model for multimodal vision-robot navigation. *Appl Intell* 53(20):23244–23270. <https://doi.org/10.1007/s10489-023-04754-7>
- Rais MS, Boudour R, Zouaidia K et al (2023) Decision making for autonomous vehicles in highway scenarios using harmonic sk deep sarsa. *Appl Intell* 53(3):2488–2505. <https://doi.org/10.1007/s10489-022-03357-y>
- Hu J, Li X, Hu W et al (2024) A cooperative control methodology considering dynamic interaction for multiple connected and automated vehicles in the merging zone. *IEEE Trans Intell Transp Syst* 25(9):12669–12681. <https://doi.org/10.1109/TITS.2024.3386200>
- Hu J, Li X, Cen Y et al (2022) A roadside decision-making methodology based on deep reinforcement learning to simultaneously improve the safety and efficiency of merging zone. *IEEE Trans Intell Transp Syst* 23(10):18620–18631. <https://doi.org/10.1109/TITS.2022.3157910>
- Kahn G, Abbeel P, Levine S (2021) Badgr: An autonomous self-supervised learning-based navigation system. *IEEE Robot Autom Lett* 6(2):1312–1319. <https://doi.org/10.1109/LRA.2021.3057023>
- Hu H, Zhang K, Tan AH et al (2021) A sim-to-real pipeline for deep reinforcement learning for autonomous robot navigation in cluttered rough terrain. *IEEE Robot Autom Lett* 6(4):6569–6576. <https://doi.org/10.1109/LRA.2021.3093551>
- Josef S, Degani A (2020) Deep reinforcement learning for safe local planning of a ground vehicle in unknown rough terrain. *IEEE Robot Autom Lett* 5(4):6748–6755. <https://doi.org/10.1109/LRA.2020.3011912>
- Zhu W, Hayashibe M (2023) A hierarchical deep reinforcement learning framework with high efficiency and generalization for fast and safe navigation. *IEEE Trans Ind Electron* 70(5):4962–4971. <https://doi.org/10.1109/TIE.2022.3190850>
- Zhang K, Niroui F, Ficocelli M et al (2018) Robot navigation of environments with unknown rough terrain using deep reinforcement learning. In: 2018 IEEE International Symposium on Safety, Security, and Rescue Robotics (SSRR), IEEE, pp 1–7. <https://doi.org/10.1109/SSRR.2018.8468643>
- Cimurs R, Suh IH, Lee JH (2022) Goal-driven autonomous exploration through deep reinforcement learning. *IEEE Robot Autom Lett* 7(2):730–737. <https://doi.org/10.1109/LRA.2021.3133591>
- Weerakoon K, Sathyamoorthy AJ, Patel U et al (2022) Terp: Reliable planning in uneven outdoor environments using deep reinforcement learning. In: 2022 International Conference on Robotics and Automation (ICRA), pp 9447–9453. <https://doi.org/10.1109/ICRA46639.2022.9812238>
- Samsani SS, Mutahira H, Muhammad MS (2022) Memory-based crowd-aware robot navigation using deep reinforcement learning. *Complex Intell Syst* 9:2147–2158. <https://doi.org/10.1007/s40747-022-00906-3>
- Huang W, Zhou Y, He X et al (2024) Goal-guided transformer-enabled reinforcement learning for efficient autonomous navigation. *IEEE Trans Intell Transp Syst* 25(2):1832–1845. <https://doi.org/10.1109/TITS.2023.3312453>
- Zhou K, Guo C, Zhang H et al (2023) Optimal graph transformer viterbi knowledge inference network for more successful visual navigation. *Adv Eng Inf* 55:101889. <https://doi.org/10.1016/j.aei.2023.101889>
- Yang Y, Jiang J, Zhang J et al (2023) St²: Spatial-temporal state transformer for crowd-aware autonomous navigation. *IEEE Robot Autom Lett* 8(2):912–919. <https://doi.org/10.1109/LRA.2023.3234815>
- Liu H, Huang Z, Mo X et al (2024) Augmenting reinforcement learning with transformer-based scene representation learning for decision-making of autonomous driving. *IEEE Trans Intell Veh* 9(3):4405–4421. <https://doi.org/10.1109/TIV.2024.3372625>
- Miranda VRF, Neto AA, Freitas GM et al (2024) Generalization in deep reinforcement learning for robotic navigation by reward shaping. *IEEE Trans Ind Electron* 71(6):6013–6020. <https://doi.org/10.1109/TIE.2023.3290244>
- Wu J, Zhou Y, Yang H et al (2023) Human-guided reinforcement learning with sim-to-real transfer for autonomous navigation. *IEEE Trans Pattern Anal Mach Intell* 45(12):14745–14759. <https://doi.org/10.1109/TPAMI.2023.3314762>
- Zhang L, Peng J, Yi W et al (2024) A state-decomposition ddpg algorithm for uav autonomous navigation in 3-d complex environments. *IEEE Internet Things J* 11(6):10778–10790. <https://doi.org/10.1109/JIOT.2023.3327753>
- Lim H, Oh M, Myung H (2021) Patchwork: Concentric zone-based region-wise ground segmentation with ground likelihood estimation using a 3d lidar sensor. *IEEE Robot Autom Lett* 6(4):6458–6465. <https://doi.org/10.1109/LRA.2021.3093009>
- Vaswani A, Shazeer N, Parmar N et al (2017) Attention is all you need. In: Guyon I, Luxburg UV, Bengio S et al (eds) *Advances in Neural Information Processing Systems*, Long Beach, CA, USA
- Han SY, Liang T (2022) Reinforcement-learning-based vibration control for a vehicle semi-active suspension system via the ppo approach. *Appl Sci* 12(6):3078. <https://doi.org/10.3390/app12063078>
- Huang X, Deng H, Zhang W et al (2021) Towards multi-modal perception-based navigation: A deep reinforcement learning

- method. *IEEE Robot Autom Lett* 6(3):4986–4993. <https://doi.org/10.1109/LRA.2021.3064461>
34. Raffin A, Hill A, Gleave A et al (2021) Stable-baselines3: Reliable reinforcement learning implementations. *J Mach Learn Res* 22(268):1–8. <http://jmlr.org/papers/v22/20-1364.html>
 35. Teng S, Hu X, Deng P et al (2023) Motion planning for autonomous driving: The state of the art and future perspectives. *IEEE Trans Intell Veh* 8(6):3692–3711. <https://doi.org/10.1109/TIV.2023.3274536>
 36. Dosovitskiy A, Ros G, Codevilla F et al (2017) CARLA: An open urban driving simulator. In: *Proceedings of the 1st annual conference on robot learning, proceedings of machine learning research*, vol 78. PMLR, pp 1–16. <https://proceedings.mlr.press/v78/dosovitskiy17a.html>
 37. Yang F, Cao C, Zhu H et al (2022) Far planner: Fast, attemptable route planner using dynamic visibility update. In: *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp 9–16. <https://doi.org/10.1109/IROS47612.2022.9981574>
 38. Tordesillas J, Lopez BT, Everett M et al (2022) Faster: Fast and safe trajectory planner for navigation in unknown environments. *IEEE Trans Robot* 38(2):922–938. <https://doi.org/10.1109/TRO.2021.3100142>
 39. Benatti S, Young A, Elmquist A et al (2022) End-to-end learning for off-road terrain navigation using the chrono open-source simulation platform. *Multibody Syst Dyn* 54(4):399–414. <https://doi.org/10.1007/s11044-022-09816-1>
 40. Wang Y, Wang J, Yang Y et al (2022) An end-to-end deep reinforcement learning model based on proximal policy optimization algorithm for autonomous driving of off-road vehicle. In: *International conference on autonomous unmanned systems*. Springer, pp 2692–2704. https://doi.org/10.1007/978-981-99-0479-2_248
 41. Abbaszadeh Shahri A, Chunling S, Larsson S (2024) A hybrid ensemble-based automated deep learning approach to generate 3d geo-models and uncertainty analysis. *Eng Comput* 40:1501–1516. <https://doi.org/10.1007/s00366-023-01852-5>
 42. Abbaszadeh Shahri A, Shan C, Larsson S (2022) A novel approach to uncertainty quantification in groundwater table modeling by automated predictive deep learning. *Nat Resour Res* 31:1351–1373. <https://doi.org/10.1007/s11053-022-10051-w>
 43. Gu Y, Cheng Y, Chen CLP et al (2022) Proximal policy optimization with policy feedback. *IEEE Trans Syst Man Cybern Syst* 52(7):4600–4610. <https://doi.org/10.1109/TSMC.2021.3098451>
 44. Hiraoka T, Imagawa T, Hashimoto T et al (2022) Dropout q-functions for doubly efficient reinforcement learning. In: *International conference on learning representations*. <https://openreview.net/forum?id=xCVJMsPv3RT>
 45. Bhatt A, Palenicek D, Belousov B et al (2024) Crossq: Batch normalization in deep reinforcement learning for greater sample efficiency and simplicity. In: *The twelfth international conference on learning representations, Vienna, Austria*. <https://openreview.net/forum?id=PczQtTsTIX>
 46. Fujimoto S, Chang WD, Smith E et al (2023) For sale: State-action representation learning for deep reinforcement learning. In: *Advances in neural information processing systems*, New Orleans, LA, USA, pp 61573–61624. https://proceedings.neurips.cc/paper_files/paper/2023/file/c20ac0df6c213db6d3a930fe9c7296c8-Paper-Conference.pdf
 47. Abbaszadeh Shahri A, Shan C, Larsson S et al (2024) Normalizing large scale sensor-based mwd data: An automated method toward a unified database. *Sensors* 24(4):1209. <https://doi.org/10.3390/s24041209>
 48. Yan X, Yang J, Zhu X et al (2024) Denoising framework based on multiframe continuous point clouds for autonomous driving lidar in snowy weather. *IEEE Sens J* 24(7):10515–10527. <https://doi.org/10.1109/JSEN.2024.3358341>
 49. Wu J, Huang Z, Huang W et al (2024) Prioritized experience-based reinforcement learning with human guidance for autonomous driving. *IEEE Trans Neural Netw Learn Syst* 35(1):855–869. <https://doi.org/10.1109/TNNLS.2022.3177685>
 50. Wu J, Huang Z, Hu Z et al (2023) Toward human-in-the-loop ai: Enhancing deep reinforcement learning via real-time human guidance for autonomous driving. *Engineering* 21:75–91. <https://doi.org/10.1016/j.eng.2022.05.017>

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.



Zhijian Li received his bachelor's degree from China University of Geosciences (Beijing) in 2022. He is currently pursuing a master's degree in the School of Instrument Science and Engineering at Southeast University. His research interests include path planning and reinforcement learning.



Xu Li (Member, IEEE) received the Ph.D. degree in instrument science and technology from Southeast University, Nanjing, China, in 2006. He is currently a Professor with the School of Instrument Science and Engineering, Southeast University. Since 2002, he has been involved in multisensor integration navigation and control for automated vehicles. His current research interests include sensor integration navigation, information fusion, automated vehicles, and intelligent transportation system.



Jinchao Hu received the Ph.D. degree at Southeast University, Nanjing, Chin, in 2023. He is currently with the School of Information Engineering, Ningxia University, Yinchuan, China. His research interests include Connected and Automated Vehicles (CAVs) and Intelligent Vehicle-Infrastructure System (IVIS). In addition, he has published several papers in IEEE Trans journals such as TITS, TII, and TIV, and has served as a reviewer for TII.



Xixiang Liu received the Ph.D. degree in instrument science and technology from the Southeast University, Nanjing, China, in 2007. He is currently a professor with the School of Instrument Science and Engineering, Southeast University. His research interests include inertial navigation, integrated navigation and information fusion technology.